

HERRAMIENTAS Y ENTORNOS DE PROGRAMACIÓN

Tema 3. Entornos de Desarrollo. Caso de Estudio:
Tecnología .NET

Escuela Superior de Informática



Ramón Hervás Lucas - Curso 2009/2010 - HyEP

1

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo.

- **Entornos de Desarrollo. Caso de Estudio
Tecnología .NET (~ 8 horas)**

- **Características generales de .NET**
- Ensamblados (Assemblies)
- Administración de datos con ADO.NET
- .NET frente a otras tecnologías
- El entorno Visual Studio .NET
- Lenguaje de Programación C#

2

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **¿Qué es .NET?**

- .NET es una plataforma para el desarrollo, despliegue y ejecución de aplicaciones orientadas a servicios sobre entornos altamente distribuidos.
- Es el Resultado de la confluencia de dos proyectos:
 - El primero de ellos tenía como objetivo la mejora del desarrollo sobre las plataformas Windows, prestando una especial atención a la mejora del modelo COM.
 - El segundo proyecto, conocido como NGWS (*Next Generation Windows Services*), tenía como objetivo la creación de una plataforma para el desarrollo del software como servicio.
- La plataforma .NET cubre todas las capas del desarrollo de software, existiendo una alta integración entre las tecnologías de presentación, de componentes y de acceso a datos.

3

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Objetivos de la Tecnología .NET**

- Proporcionar un modelo de programación simple y consistente.
 - A diferencia del modelo anterior, en el cual algunas facilidades del sistema operativo son ofrecidas mediante DLLs y otras mediante objetos COM, todos los servicios del *framework* son proporcionados de la misma forma mediante un modelo de programación orientado a objetos.
 - Así mismo, se ha simplificado el modelo de programación, lo que permite a los desarrolladores centrarse en las cuestiones relativas a la lógica de la aplicación.
- Liberar al programador de las cuestiones de infraestructura (aspectos no funcionales).
 - El *framework* .NET se encarga de gestionar automáticamente tales cuestiones como la gestión de la memoria, de los hilos o de los objetos remotos.

4

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Objetivos de la Tecnología .NET**
 - **Proporcionar integración entre diferentes lenguajes.** El problema de la interoperabilidad ha sido considerado durante muchos años, desarrollándose varios estándares y arquitecturas con diferente nivel de éxito:
 - **Estándares de representación de datos**, que solucionan las cuestiones relativas al paso de tipos de datos entre distintas máquinas, tales como los formatos *little-endian* y *big-endian*.
 - **Estándares arquitecturales**, como RPC, CORBA o COM, que solucionan las cuestiones relativas a la llamada de métodos entre diferentes lenguajes, procesos o máquinas.
 - **Estándares de lenguajes**, como ANSI C, que permite la distribución de código fuente entre distintos compiladores y máquinas.
 - **Entornos de ejecución**, como los proporcionados por las máquinas virtuales de SmallTalk y Java, que permiten la ejecución en diferentes máquinas físicas proporcionando un entorno de ejecución estandarizado.

Sin embargo, ninguno de estos esquemas ha solucionado completamente los problemas asociados con un entorno de computación distribuido.

5

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Objetivos de la Tecnología .NET**
 - **Proporcionar una ejecución multiplataforma.**
 - .NET ha sido diseñado para ser independiente de la plataforma sobre la cual se ejecutaran las aplicaciones. Para conseguir este objetivo las aplicaciones .NET se compilan a un lenguaje intermedio denominado Lenguaje Intermedio de Microsoft o MSIL (*Microsoft Intermediate Language*), el cual es independiente de las instrucciones de una CPU concreta.
 - **Proporcionar soporte para arquitecturas fuertemente acopladas y débilmente acopladas.**
 - Para conseguir un buen rendimiento, escalabilidad y confiabilidad con grandes sistemas distribuidos, hay operaciones en las cuales los componentes están fuertemente acoplados.
 - Sin embargo, también debe soportarse una comunicación débilmente acoplada, de forma que una transacción no quede interrumpida o bloqueada por cualquier dependencia en tiempo de ejecución.

6

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Objetivos de la Tecnología .NET**
 - **Proporcionar un mecanismo de errores consistente.**
 - En la plataforma Windows no existe un sistema unificado para el manejo de los errores, de forma que este se realiza mediante códigos de error Win32, mediante la variable HRESULT en COM, o mediante el lanzamiento de excepciones. En .NET todos los errores son manejados mediante un mecanismo de excepciones, el cual permite aislar el código de manejo de errores del resto, permitiéndose la propagación de excepciones entre distintos módulos y lenguajes.
 - **Proporcionar un mecanismo de seguridad avanzado.**
 - Así, la plataforma.NET proporciona un modelo de seguridad basado en la evidencia, que posee un modelo de control de gran granularidad, pudiendo basarse o no en quien escribió el código, que intenta hacer dicho código, donde está instalado, y quién está intentando ejecutar dicho código.
 - **Sistema de despliegue simple.**
 - Se ha eliminado la necesidad de tratar con el registro, con GUIDs, etc, de forma que la instalación de una aplicación es tan sencilla como su copia en un directorio.

7

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

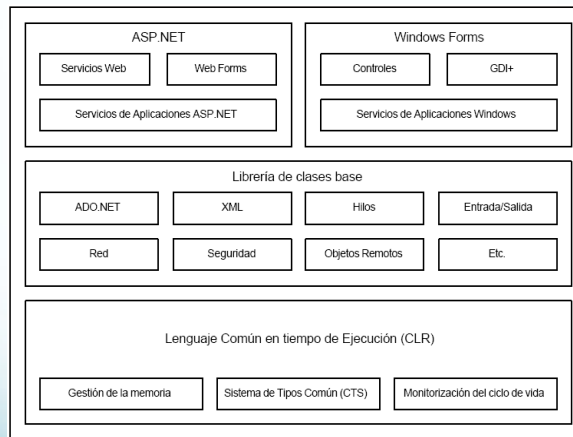
- **Compatibilidad de Visual Basic.NET**
 - Visual Basic.NET NO es 100% compatible con las versiones anteriores
 - **Alternativas iniciales**
 - Mejorar el código base de Visual Basic para que se ejecute sobre .NET
 - Reconstruir Visual Basic desde cero para aprovechar todas las posibilidades de .NET
 - **Objetivos alcanzados**
 - Garantía de interoperatividad con el resto de lenguajes .NET
 - Comparte tipos de variables, *arrays*, tipos definidos por el usuario, clases e interfaces que C++ y C#.
 - Visual Basic .NET es realmente un lenguaje orientado a objetos
 - **Pérdidas**
 - Eliminación de cadenas de longitud fija y *arrays* cuyo índice inicial es distinto a cero
 - Eliminación de características inconsistentes como GoSub/Return

8

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Componentes principales**



9

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- El lenguaje común en tiempo de ejecución, o CLR, es el motor de ejecución para las aplicaciones de .NET.
- El CLR puede considerarse como el núcleo de .NET, desempeñando el papel de una máquina virtual que se encarga de gestionar la ejecución del código y de proporcionar una serie de servicios a dicho código.
- Entre los **servicios proporcionados por el CLR** a las aplicaciones .NET se encuentran los siguientes:
 - Gestión del código, encargándose de la carga y ejecución del código MSIL.
 - Aislamiento de la memoria de las aplicaciones, de forma que desde el código perteneciente a un determinado proceso no pueda accederse al código o datos pertenecientes a otro proceso, lo que permite que un error en una aplicación no afecte al resto.
 - Garantizar la robustez del código mediante la implementación de un Sistema de Tipos Común o CTS (*Common Type System*).

10

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**
 - **Servicios del CLR (continuación)**
 - Conversión del código MSIL al código nativo, utilizándose para ello técnicas de compilación "Just In Time" (JIT).
 - Acceso a los metadatos, que contienen información sobre los tipos, y sus dependencias, definidos en el código.
 - Gestión automática de la memoria, encargándose de gestionar las referencias de los objetos y de la tareas de recolección de basura.
 - Asegurar la seguridad en los accesos del código a los recursos, la cual estará en función del nivel de confianza del que goce el código, lo que dependerá de una serie de factores tales como su origen.
 - Manejo de las excepciones, incluyendo las excepciones entre código escrito en diferentes lenguajes.
 - Interoperabilidad con el código no gestionado, lo que incluye desde objetos COM hasta código incluido en DLLs.
 - Soporte de servicios para los desarrolladores, tales como la depuración.

11

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**
 - El CLR es el que posibilita la integración entre diferentes lenguajes, proporcionando a su vez una mejora en el rendimiento como consecuencia de los servicios que ofrece, tales como la gestión automática de la memoria. El CLR esta formado principalmente por tres componentes:
 - **Un Sistema de Tipos Común o CTS**, formado por un amplio conjunto de tipos y operaciones que se encuentran presentes en la mayoría de los lenguajes de programación.
 - **Un sistema de metadatos**, que permite almacenar dichos metadatos junto con los tipos a los que se refieren en tiempo de compilación, así como obtenerlos en tiempo de ejecución.
 - **Un sistema de ejecución**, que se encarga de ejecutar las aplicaciones del framework .NET, haciendo uso del sistema de información de metadatos para desarrollar los servicios tales como la gestión de la memoria.

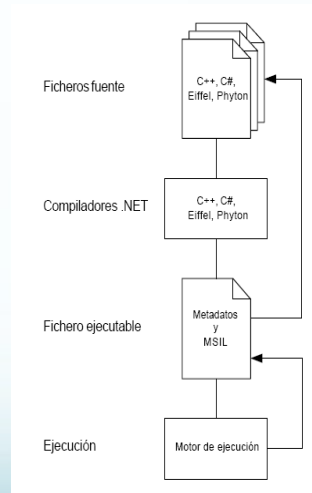
12

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- Un fichero fuente, podría contener una definición de un nuevo tipo escrito en cualquiera de los lenguajes soportados por .NET. Ese tipo podría heredar de cualquiera de los tipos de las librerías de .NET.
- Dicho fichero es compilado, generando un fichero con código intermedio MSIL y con los metadatos correspondientes a dicho tipo.
- Los metadatos podrían ser utilizados para importar dicho tipo, de forma que pueda ser utilizado por cualquiera de los lenguajes de .NET
- En tiempo de ejecución, el sistema carga el fichero con MSIL, compila a código máquina. Cualquier referencia a un tipo situado en un fichero de MSIL diferente provoca que dicho fichero sea cargado y leídos sus metadatos, siguiéndose el mismo proceso de ejecución.



13

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Sistema de tipos común (CTS)**

Para conseguir la interoperabilidad entre lenguajes es necesario adoptar un sistema de tipos común. Así, el sistema de tipos común (CTS) define como se declaran, utilizan y gestionan los tipos en el CLR. El CTS desarrolla las siguientes funciones:

- Establece un framework que permite la integración entre lenguajes, la seguridad de tipos, y la ejecución de código con un alto rendimiento.
- Proporciona un modelo orientado a objetos que soporta la implementación de muchos lenguajes de programación.
- Define una serie de reglas que los lenguajes deben seguir para permitir la interoperabilidad de los mismos.

14

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Sistema de tipos común (CTS)**

Tipos Valor.

- Las instancias de los tipos Valor son almacenadas como la representación de su valor como una secuencia de bits en memoria, careciendo del concepto de identidad.
- Dentro de los tipos valor se encuentran los predefinidos (implementados por el CLR), los definidos a medida por el usuario, y las enumerados.

Tipos Referencia.

- Las instancias de los tipos Referencia son almacenadas como referencias a la localización de su valor.
- Los tipos referencia son una combinación de una localización, su identidad, y una secuencia de bits (su valor).
- Dentro de los tipos referencia se encuentran los tipos interfaz, los tipos punteros, y los tipos autodescriptivos. Los tipos autodescriptivos son aquellos en los cuales es posible obtener el tipo de su valor por inspección.

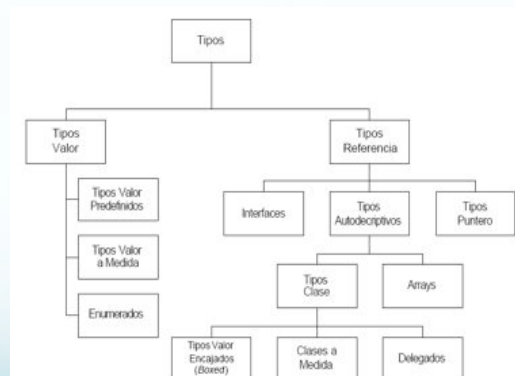
15

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Sistema de tipos común (CTS)**



16

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Sistema de tipos común (CTS). Definición de Tipos.**

Una definición de un tipo construye un nuevo tipo a partir de tipos existentes. Los tipos valor predefinidos, los punteros, arrays y delegados son definidos al ser utilizados, por lo que a estos tipos se les conoce como tipos implícitos. La definición de un tipo incluye los siguientes elementos:

- Los atributos definidos sobre el tipo (cómo se verá en la sección de los metadatos, los atributos son un mecanismo de extensión de los mismos).
- La visibilidad del tipo. Un tipo puede ser visible a todos los ensamblados (visibilidad pública), o sólo para el ensamblado que lo define (visibilidad de ensamblado).
- Nombre del tipo. Un tipo queda definido dentro de un ensamblado, por lo que sólo tiene que ser único dentro del ensamblado.
- El tipo base del tipo definido. Un tipo definido sólo puede tener un tipo base. Las interfaces implementadas por el tipo.

...Continúa...

17

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Sistema de tipos común (CTS). Definición de Tipos.**

Las definiciones de cada uno de los miembros del tipo. Dentro de un tipo pueden definirse los siguientes miembros:

- **Eventos.** Definen incidentes a los que se puede responder.
- **Campos** (variables). Describen y contienen el valor de un tipo.
- **Tipos anidados.** Definen a un tipo dentro del ámbito del tipo que lo contiene.
- **Métodos.** Definen las operaciones disponibles para un tipo
- **Propiedades.** Nombran a un valor lógico o al estado de un tipo, y constituyen una alternativa a los tradicionales métodos de acceso/modificación get/set, de forma que internamente las propiedades son mapeadas a métodos get y set. Las propiedades pueden contener lógica interna, así como lanzar excepciones si fuera necesario.

18

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- Lenguaje común en tiempo de ejecución
 - Sistema de tipos común (CTS). Definición de Tipos.
 - Un Ejemplo:

```
private string dni;  
...  
public string ENI  
{  
    get  
    {  
        return dni;  
    }  
    set  
    {  
        if (value.Length == 0)  
        {  
            dni = value;  
        }  
        else  
        {  
            throw new INIFormatoException();  
        }  
    }  
}
```

19

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- Lenguaje común en tiempo de ejecución
 - Sistema de tipos común (CTS). Tipos Referencia.

Los tipos referencia son la combinación de una localización, y una secuencia de bits. Las localizaciones, que denotan las áreas de memoria en las cuales los valores pueden ser almacenados, poseen seguridad de tipos, de forma que sólo pueden asignarse tipos compatibles. A continuación se describen los distintos tipos Referencia del CTS.

- **Clases** Como en cualquier sistema orientado a objetos, el CTS incluye el concepto de clase. Implícitamente, cualquier clase hereda de System.Object, la cual proporciona una serie de métodos.
- **Delegados** El CTS soporta un tipo de objetos denominados delegados, los cuales tienen una finalidad similar a los punteros a funciones de C++, pero con la diferencia en que estos cuentan con la seguridad del sistema de tipos, de forma que siempre apuntan a un objeto válido.
- **Arrays** Los arrays son definidos especificando el tipo de sus elementos, su número de dimensiones y sus límites inferior y superior para cada dimensión.
- **Interfaces** Un tipo interfaz es la especificación parcial de un tipo, actuando como contratos que ligan a los implementadores con lo especificado en la interfaz.
- **Punteros** El CTS soporta tres tipos de punteros: punteros gestionados, punteros no gestionados, y punteros no gestionados a funciones.

20

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Metadatos**

- Los metadatos son información binaria que describe los tipos implementados por un programa.
- Los metadatos se almacenan en un fichero Ejecutable Portable (PE) o en memoria, de forma que cuando un fichero con código es compilado, los metadatos son almacenados junto con el código MSIL. Todos los compiladores para .NET están obligados a emitir metadatos sobre cada tipo contenido en un fichero fuente.
- Sirven de puente que enlaza el sistema de tipos común (CTS) y el motor de ejecución del .NET.
- Los metadatos solucionan dos de los problemas existentes en muchos de los sistemas actuales basados en componentes, como son que la información sobre los componentes, como los ficheros IDL, son almacenados separados de los componentes, y que la descripción de los componentes que poseen muchos de estos sistemas sólo especifican la sintaxis de sus interfaces, y no su semántica.
- En .NET se ha solucionado este problema proporcionando un mecanismo de extensión de los metadatos, conocido como atributos.

21

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Metadatos**

.NET almacenan el código MSIL junto con los metadatos, constituyendo así unas unidades autodescriptivas denominadas ensamblados, mediante los cuales se simplifica enormemente el despliegue de las aplicaciones del framework .NET. Debido a su importancia, los ensamblados serán explicados más adelante en un apartado específico.

Los metadatos proporcionan los siguientes beneficios:

- Proporcionan ficheros de código autodescriptivos, eliminando la necesidad del registro y manteniéndose siempre sincronizados las descripciones de los tipos y el código que los implementan.
- Proporcionan la información necesaria para conseguir la interoperabilidad entre distintos lenguajes.
- Proporcionan la información necesaria que requiere el sistema de ejecución para la gestión de los objetos. Así mismo, los metadatos permiten las invocaciones remotas en la plataforma .NET.
- Mediante los atributos es posible especificar una serie de aspectos que permiten especificar más en detalle como se comporta un programa en tiempo de ejecución.

22

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Sistema de Ejecución**

El motor de ejecución del CLR es el responsable de asegurar que el código es ejecutado como requiere, proporcionando una serie de facilidades para el código MSIL como:

- Carga del código y verificación.
- Gestión de las excepciones.
- Compilación "Just In Time" (JIT).
- Gestión de la memoria.
- Seguridad

Lenguaje intermedio MSIL: El código intermedio MSIL generado por los compiladores del framework .NET es independiente del juego de instrucciones de una CPU específica. La principal ventaja del MSIL es que proporciona una capa de abstracción del hardware, lo que facilita la ejecución multiplataforma y la integración entre lenguajes

23

Herramientas y Entornos de Programación

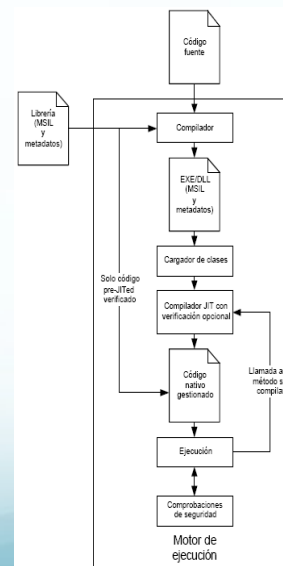
Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**

- **Sistema de ejecución**

Compilador JIT: La traducción de MSIL a código nativo de la CPU es realizada por un compilador "Just In Time" o *jitter*,

- El Jitter va convirtiendo dinámicamente el código MSIL a ejecutar en código nativo según sea necesario.
- La compilación JIT tiene en cuenta el hecho de que algunas porciones de código no serán llamadas durante la ejecución, por lo que en lugar de invertir tiempo y memoria en convertir todo el código MSIL a código nativo, únicamente convierte el código que es necesario durante la ejecución, almacenándolo por si fuera necesario en futuras llamadas.



Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Lenguaje común en tiempo de ejecución**
 - **Sistema de ejecución. Recolector de basura:**
 - El recolector de basura es el responsable de eliminar los objetos de la memoria *heap* que no van a ser referenciados nunca más, compactando el resto de objetos, y actualizando tras esto la referencia a la última posición de memoria libre.
 - El proceso de recolección de basura puede ser lanzado automáticamente por el CLR o por una aplicación que lo invoca explícitamente
 - Para averiguar qué objetos no van a ser referenciados nunca más, el recolector de basura obtiene las referencias “raíces”, que son aquellos objetos referenciados directamente por la aplicación. El recolector obtiene a su vez los objetos referenciados por cada referencia “raíz”, y así sucesivamente. Tras este proceso el recolector de basura es libre de eliminar los objetos no válidos

25

Herramientas y Entornos de Programación

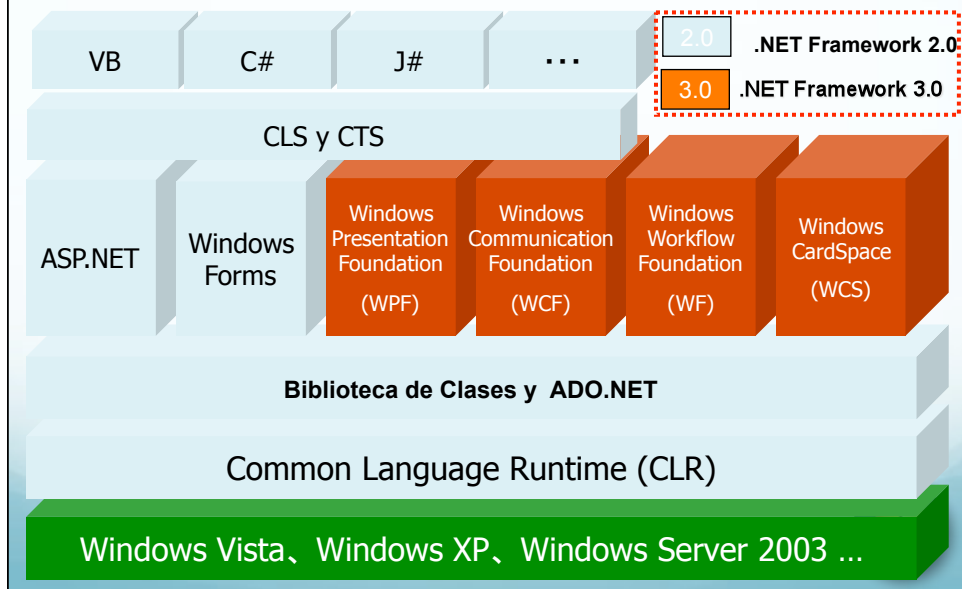
Tema 3. Entornos de Desarrollo. Características

- **Especificación de Lenguaje Común (CLS)**
 - El CLR proporciona, mediante el sistema de tipos común CTS y los metadatos, la infraestructura necesaria para lograr la interoperabilidad entre lenguajes
 - Todos los lenguajes siguen las reglas definidas en el CTS para la definición y el uso de los tipos, y los metadatos definen un mecanismo uniforme para el almacenamiento y recuperación de la información sobre dichos tipos.
 - A pesar de esto, no hay ninguna garantía de que la funcionalidad de los tipos escritos por un desarrollador en un lenguaje determinado pueda ser completamente utilizado por otros desarrolladores que utilizan otros lenguajes.
 - Para asegurar que el código escrito en un lenguaje sea accesible desde otros lenguajes se ha definido la Especificación del Lenguaje Común o CLS (Common Language Specification), que establece el conjunto mínimo de características que deben soportarse para asegurar la interoperabilidad, siendo dicho conjunto de características mínimas un subconjunto del CTS.
 - El CLS ha sido diseñado para ser lo suficientemente grande como para que incluya las construcciones que son utilizadas comúnmente en los lenguajes, y lo suficientemente pequeño para que la mayoría de los lenguajes puedan cumplirlo.

26

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características



Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Entornos de Desarrollo. Caso de Estudio**
Tecnología .NET (~ 8 horas)

- Características generales de .NET
- **Ensamblados (Assemblies)**
- Administración de datos con ADO.NET
- .NET frente a otras tecnologías
- El entorno Visual Studio .NET
- Lenguaje de Programación C#

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Introducción a los Ensamblados**

- Los ensamblados son los bloques de construcción de las aplicaciones para la plataforma .NET, siendo la unidad fundamental de despliegue, de re-uso y de control de versiones.
- Un ensamblado es una colección de tipos y recursos que constituyen una unidad lógica de funcionalidad, proporcionando la información que el CLR necesita sobre las implementaciones de dichos tipos.
- Los ensamblados pueden clasificarse atendiendo a varios criterios. Así, los ensamblados pueden ser:
 - **Ensamblados estáticos o dinámicos:** los ensamblados estáticos son generados en tiempo de compilación y almacenados a disco, mientras que los dinámicos son generados en tiempo de ejecución (mediante los servicios de reflexión), ejecutados directamente desde memoria y pueden ser salvados a disco una vez que han sido ejecutados.
 - **Ensamblados multifichero o con un único fichero**
 - **Ensamblados privados o compartidos:** los ensamblados privados son aquellos que son utilizados únicamente por la aplicación con la cual han sido desplegados, mientras que un ensamblado compartido puede ser utilizado por varias aplicaciones.

29

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Características de los Ensamblados**

- Contiene el código intermedio (MSIL) que será ejecutado por el *runtime*, así como los metadatos generados por el compilador y el manifiesto del ensamblado. Los ensamblados son unidades autodescriptivas, eliminándose toda dependencia con el registro de Windows, lo que permite simplificar el despliegue de los mismos.
- Define una frontera de encapsulación para los tipos que contiene. La identidad de un tipo queda definido, en parte, por el ensamblado al que pertenece, de forma que dos tipos con idéntico nombre definidos en ensamblados diferentes son considerados independientes.
- Constituye una frontera del ámbito de las referencias. El manifiesto del ensamblado contiene metadatos que son utilizados para la obtención de los tipos y recursos solicitados, especificando los tipos y recursos expuestos por el ensamblado así como los ensamblados de los cuales depende.
- Constituye la unidad mínima versionable. La política de versiones es aplicada sobre todos los tipos y recursos contenidos en el ensamblado. La política de versiones asegura que es cargado el ensamblado correcto ante la invocación de un ensamblado

30

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Características de los Ensamblados**

- Constituye la unidad de despliegue. Al arrancar una aplicación, solo los ensamblados que son llamados inicialmente tienen que estar presentes. El resto de ensamblados pueden ser obtenidos bajo demanda.
- Permite el aislamiento de las aplicaciones. La existencia de ensamblados privados favorecen el aislamiento de las aplicaciones, de forma que los cambios realizados en una aplicación no afecten al comportamiento del resto.
- Definen un contexto de seguridad. En la arquitectura .NET, las medidas de seguridad son tomadas a nivel de los ensamblados, quedando definidas mediante los metadatos del ensamblado, concretamente en su manifiesto.
- Soportan la ejecución de múltiples versiones simultáneas (*side-by-side execution*). El *runtime* tiene la capacidad de ejecutar múltiples versiones del mismo ensamblado en una única máquina, permitiendo aislar versiones incompatibles de un mismo ensamblado y simplificar la actualización de los mismos.

31

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Estructura de un Ensamblado**

- En general, la estructura lógica de un ensamblado estático consta de cuatro elementos:
 - El manifiesto del ensamblado, que contiene metadatos del ensamblado.
 - Los metadatos que describen los tipos del ensamblado.
 - El código en lenguaje intermedio (MSIL) que implementa los tipos.
 - Un conjunto de recursos.
- Estos cuatro elementos lógicos pueden estar dispuestos físicamente de varias formas, de manera que esto nos conduce a la posibilidad de tener ensamblados de un único fichero y ensamblados multifichero. Estos ficheros físicos, cuando su contenido es metadatos y, opcionalmente, código intermedio MSIL, son denominados módulos.

32

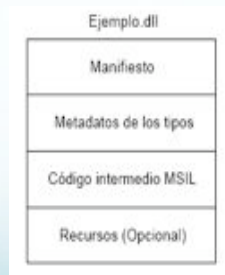
Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Estructura de un Ensamblado**

Ensamblados de un solo fichero

Un ensamblado puede estar formado por un único módulo, estableciéndose en este caso una correspondencia uno a uno entre el ensamblado (punto de vista lógico) y el fichero binario (punto de vista físico). Un ensamblado de un solo fichero tiene una estructura como la mostrada en la siguiente figura:



33

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Estructura de un Ensamblado**

Ensamblados multifichero

- Un ensamblado puede estar compuesto por una serie de ficheros físicos, de forma que los elementos lógicos del ensamblado se encuentran distribuidos en una serie de módulos o en ficheros de recursos.
- En cada ensamblado solo puede haber un módulo que contenga el manifiesto, mientras que el resto de módulos solo pueden contener metadatos sobre los tipos y opcionalmente código intermedio.
- Una de las ventajas de la utilización de los ensamblados multifichero es la optimización de la descarga de un ensamblado, de forma que situando los tipos o recursos que son poco utilizados en módulos separados la descarga del ensamblado requiere una transferencia de datos, descargándose el resto de módulos únicamente en caso de que sean referenciados.
- Los módulos que componen un ensamblado multifichero están relacionados lógicamente entre sí por medio de la información contenida en el manifiesto del ensamblado, en el cual se referencia a los ficheros físicos que componen el ensamblado.

34

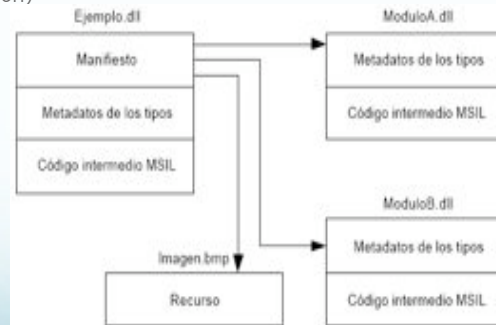
Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Estructura de un Ensamblado**

Ensamblados multifichero

En la siguiente figura se muestra un ejemplo de ensamblado multifichero, el cual está compuesto por tres módulos (el que contiene el manifiesto y los dos restantes) y por un fichero de recursos (que contiene una imagen)



35

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Manifiesto de un Ensamblado**

- El manifiesto del ensamblado contiene un conjunto de metadatos que describe como los elementos contenidos en el ensamblado están relacionados. Un manifiesto puede ser almacenado en un fichero portable (un .EXE o un .DLL) junto metadatos de los tipos y código intermedio MSIL o en un fichero portable que contiene únicamente el manifiesto (esto puede darse en los ensamblados multifichero).
- Específicamente, el manifiesto de un ensamblado contiene los siguientes datos sobre el ensamblado:
 - **Identidad.** La identidad de un ensamblado está compuesta por tres partes: un nombre, un número de versión y la cultura del ensamblado (información sobre la cultura o lenguaje soportado por el ensamblado).
 - **Lista de ficheros del ensamblado.** Se incluye una lista con todos los ficheros que constituyen el ensamblado. Para cada fichero, el ensamblado almacena su nombre y un *hash* criptográfico con el contenido del fichero en el momento de la construcción del ensamblado, verificándose dicho *hash* en tiempo de ejecución para verificar que la unidad de despliegue es consistente.

...Continúa...

36

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Manifiesto de un Ensamblado**

- **Información sobre los ensamblados referenciados.** Se almacena una lista con los ensamblados referenciados de los cuales se depende estáticamente. La información de cada dependencia esta formada por la identificación del ensamblado referenciado, la cual incluye un número de versión, que es utilizado para asegurar en tiempo de ejecución que es cargado la versión correcta del ensamblado referenciado.
- **Información sobre los tipos y recursos exportados.** Contiene información relativa al mapeo entre un tipo y el fichero físico que contiene sus metadatos y su implementación, lo cual es utilizado en tiempo de ejecución. Así mismo, también contiene las opciones de visibilidad de los tipos, los cuales pueden ser visibles solo dentro del ensamblado o visible para los consumidores fuera del ensamblado.
- **Permisos solicitados.** Los permisos solicitados por un ensamblado se agrupan en tres conjuntos: aquellos que son requeridos por el ensamblado para ejecutarse, los que son deseables que tenga el ensamblado (pero que sin ellos el ensamblado mantendrá alguna funcionalidad) y los que el autor del ensamblado nunca quiere que le sean concedidos a éste.

37

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Clases de Ensamblados**

- Existen dos tipos de ensamblados: los privados y los compartidos.
- Esta clasificación es bastante débil, pues no hay diferencias en la estructura de ambos tipos de ensamblados, sino que la diferencia radica en el uso que se le da a dichos ensamblados, que pueden ser privados a una aplicación o compartidos entre varias aplicaciones.
- Las diferencias reales entre ambos tipos de ensamblados residen en las convenciones de nombrado, las políticas de versiones y en los aspectos del despliegue:

Ensamblados privados

- El **nombre** de un ensamblado debe ser único dentro de la aplicación, no existiendo la necesidad de un nombre global único.
- La política de **versiones** en el caso de los ensamblados privados es ignorada.
- Los ensamblados privados son **desplegados** en el directorio local de la aplicación o en uno de sus subdirectorios

38

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Clases de ensamblados**

Ensamblados compartidos

- **Nombrado:** Los ensamblados compartidos utilizan los denominados nombres “fuertes” para satisfacer las restricciones de nombrado asociadas a los ensamblados compartidos. Los nombres “fuertes” satisfacen 3 restricciones:
 - Garantizan la unicidad del nombre. Para lograr esto se utiliza criptografía de clave pública, la cual se basa en la utilización de un par de claves únicas, una privada y otra pública, utilizándose la clave privada para la generación del nombre del ensamblado, garantizándose su unicidad al garantizarse la unicidad de la clave privada.
 - Previenen contra la suplantación del nombrado. No es posible que alguien realice una versión de un ensamblado y que lo utilice en el proceso de carga, en lugar de utilizar la versión con la que la aplicación fue construida.
 - Proporciona una comprobación de integridad fuerte. El uso de nombres “fuertes” garantiza que los contenidos de un ensamblado no han cambiado con respecto al momento de construcción de la aplicación.

39

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Clases de ensamblados**

Ensamblados compartidos

- **Nombrado (continúa):** Un nombre compartido consta de la siguiente información:
 - Un nombre amigable (un nombre de texto) y, opcionalmente, la información relativa a la cultura del ensamblado.
 - Un número de versión.
 - Una clave pública.
 - Una firma digital.

El proceso de creación de un nombre “fuerte” es el siguiente: el autor del ensamblado, que constará de un nombre de texto, un número de versión y opcionalmente de información relativa a su cultura, firmará el fichero que contiene el manifiesto con su clave privada, incluyendo en dicho manifiesto la clave pública para que esté a disposición de los llamantes.

40

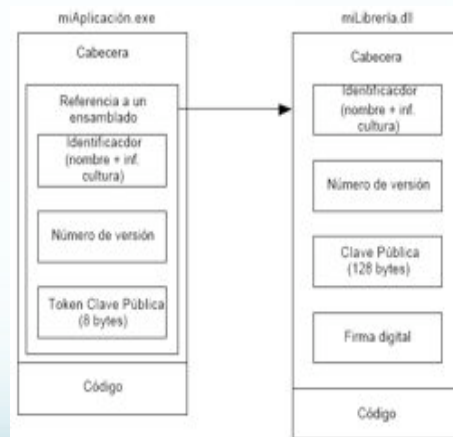
Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Clases de ensamblados**

Ensamblados compartidos

- **Nombrado (continúa):**



41

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Clases de Ensamblados**

Política de versiones:

- A diferencia de los ensamblados privados, en los cuales no se realiza un chequeo de versiones, en los ensamblados compartidos el control de las versiones es de gran importancia. La política de versiones consiste en la información de la versión contenida en el ensamblado, la cual debe ajustarse a una determinada convención, y las reglas o políticas en sí mismas.
- Información de Versión: Incluye un número de versión y una cadena de texto que añade información sobre la versión.
- Políticas: determina si se debe tomar el ensamblado por defecto u aplicar otro tipo de políticas y cargar una versión distinta que la de por defecto.

Despliegue

- Los ensamblados compartidos son, por lo general, desplegados en la caché global de ensamblados (GAC). La caché global de ensamblados es un almacenamiento general en una máquina para los ensamblados que son utilizados por mas de una aplicación.
- El mecanismo de almacenamiento de varias versiones de un mismo ensamblado es realizado automáticamente

42

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Ensamblados

- **Ejecución simultánea de múltiples versiones de Ensamblados**
 - El *runtime* posee la habilidad de ejecutar múltiples versiones de un mismo ensamblado de forma simultánea. El *runtime* del .NET permite este tipo de ejecución en la misma máquina e incluso en el mismo proceso. Los componentes que se ejecutan simultáneamente no tienen por qué mantener la compatibilidad hacia atrás, siendo esta una característica del mecanismo de manejo de versiones, y estando, por lo tanto, integrada en el *runtime* de la plataforma .NET. Hay dos tipos de ejecución de múltiples versiones simultáneas:
 - **Ejecución en la misma máquina:** múltiples versiones de una misma aplicación se ejecutan en la misma máquina sin interferirse entre sí. Las aplicaciones que soportan este tipo de ejecución deben desarrollarse cuidadosamente para evitar interferencias entre distintas versiones como consecuencia, por ejemplo, de la utilización de un recurso común.
 - **Ejecución en el mismo proceso:** para que la ejecución “codo con codo” en el mismo proceso sea posible, es necesario eliminar las dependencias estrictas sobre los recursos en el ámbito del proceso para evitar conflictos.

43

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

- **Entornos de Desarrollo. Caso de Estudio Tecnología .NET (~ 8 horas)**
 - Características generales de .NET
 - Ensamblados (Assemblies)
 - **Administración de datos con ADO.NET**
 - .NET frente a otras tecnologías
 - El entorno Visual Studio .NET
 - Lenguaje de Programación C#

44

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

- **Conceptos básicos**
 - ADO.NET es una tecnología de acceso a datos que se basa en los objetos ADO (Objetos de Datos ActiveX) anteriores.
 - También podemos decir que ADO.NET es un conjunto de clases que exponen servicios de acceso a datos al programador de .NET.
 - ADO.NET proporciona un conjunto variado de componentes para crear aplicaciones distribuidas de uso compartido de datos. Forma parte integral de .NET Framework, y proporciona acceso a datos relacionales, datos XML y datos de aplicaciones.
 - ADO.NET es compatible con diversas necesidades de programación, incluida la creación de clientes de bases de datos clientes y objetos empresariales de nivel medio utilizados por aplicaciones, herramientas, lenguajes o exploradores de Internet.
 - ADO.NET utiliza un modelo de acceso pensado para entornos desconectados. Esto quiere decir que la aplicación se conecta al origen de datos, hace lo que tiene que hacer, por ejemplo seleccionar registros, los carga en memoria y se desconecta del origen de datos.
 - ADO.NET utiliza XML como el formato para transmitir datos desde y hacia su base de datos y su aplicación Web.

45

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

- **ADO.NET**
 - ADO.NET es el conjunto de servicios proporcionados por la plataforma .NET para el acceso a las fuentes de datos. ADO.NET es la evolución de ADO, y para su construcción se ha tenido en mente a las aplicaciones de n-capas, cada vez más frecuentes, y a XML como su núcleo central.
 - El motivo que hace necesario un nuevo servicio de acceso a datos que sustituya a ADO clásico viene dado por la evolución en el desarrollo de las aplicaciones, siendo cada vez más frecuente las aplicaciones débilmente acopladas basadas en el modelo Web, las cuales utilizan XML para codificar la transmisión de sus datos.
 - Este modelo de desarrollo es bastante diferente del estilo de programación altamente acoplado, en el que las conexiones a las fuentes de datos son mantenidas durante la vida de la aplicación.

46

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

- **ADO.NET**

- Las principales ventajas de ADO.NET sobre ADO
 - Interoperabilidad,
 - Escalabilidad mejorada
 - Soporte para el tipado fuerte.
- ADO.NET ha sido diseñado para trabajar sobre conjuntos de datos desconectados, lo cual permite un procesamiento de los mismos mucho más rápido.
- ADO.NET utiliza XML como formato de transmisión de datos, lo que garantiza la interoperabilidad, pues el único requisito es que el componente que recibe los datos se ejecute en una plataforma que disponga de un parser XML,
- Se consigue eliminar la necesidad de que el receptor de los datos sea un objeto COM, tal y como sucedía con ADO.
- El modelo de objetos de ADO.NET puede ser dividido en dos niveles:
 - Nivel conectado
 - Nivel desconectado

47

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

- **Nivel Conectado de ADO.NET**

- El nivel conectado está formado por los denominados proveedores gestionados (*manager providers*), los cuales son utilizados para abrir conexiones con las bases de datos y ejecutar comandos sobre ellas.
- Este nivel es muy similar a la infraestructura existente en el ADO clásico.
- ADO.NET proporciona dos tipos de proveedores:
 - Proveedor de SQL, para los servidores SQL de Microsoft.
 - Proveedor OLEDB, para las bases de datos tales como Oracle, Access, etc.

48

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

- **Nivel Desconectado de ADO.NET**

- El nivel desconectado lo constituyen los denominados DataSets o conjuntos de datos, los cuales son la representación en memoria de un conjunto de tablas relacionadas.
- Un DataSet proporciona un modelo de objetos formado por una serie de tablas, junto con sus columnas, filas, restricciones (clave primaria, campo no nulo, etc.), así como las relaciones entre dichas tablas.
- No hay una relación estricta entre un DataSet y una fuente de datos, en el sentido en el que es posible crear una serie de tablas en un DataSet o modificar los datos de un DataSet sin que la fuente de datos tenga constancia de ellos, pues se trata de un modelo desconectado, pudiendo explícitamente en cualquier momento actualizarse el contenido de la base de datos con el contenido del DataSet.
- Un DataSet puede ser manipulado de forma programática, es decir, es posible crear tablas, su relaciones, e insertar datos utilizando una serie de objetos, tales como tablas, filas, etc.
- Así mismo, un DataSet puede ser cargado con los datos provenientes de una conexión de un proveedor gestionado, o de una fuente de datos XML.

49

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

- **Integración de ADO.NET con XML**

- La integración de ADO clásico con XML se basaba únicamente en que éste poseía una interfaz para la entrada y salida de datos en XML.
- La diferencia con respecto a ADO clásico es que ADO.NET proporciona un soporte total a XML como representación de datos, de forma que los datos obtenidos de una fuente de datos son representados internamente y transmitidos como XML.
- La convergencia entre ADO.NET y XML se produce en los DataSets. Estos tienen métodos para leer y escribir su representación en XML, por lo que cualquier dispositivo o elementos que emita datos en XML se convierte en una fuente potencial de datos para un DataSet.
- Los DataSets utilizan a un esquema XML denominado XSD (XML Schema Definition) para representar internamente la estructura de tablas y relaciones que contiene, de forma que los datos contenidos en el DataSet son codificados de acuerdo a dicho esquema.
- La representación nativa en XML de los DataSets los convierte en un medio ideal para la transferencia de datos entre las distintas capas de una aplicación.

50

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

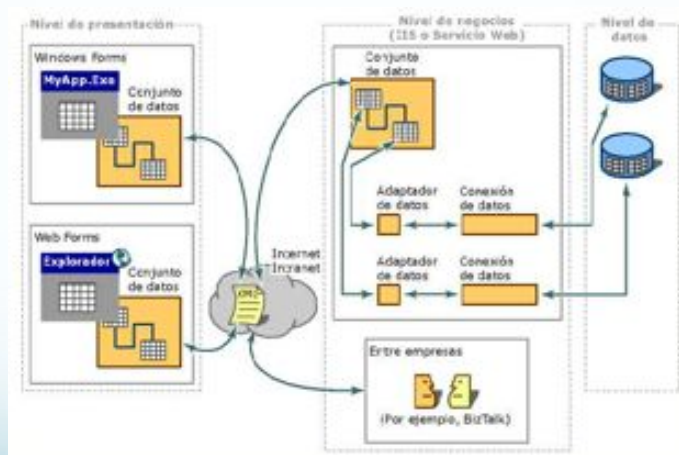
- **Espacio de Nombres para ADO.NET**

- **System.Data:** consiste en las clases que constituyen la arquitectura ADO.NET, que es el método primario para tener acceso a los datos de las aplicaciones administradas.
- **System.Data.Common:** contiene las clases que comparten los proveedores de datos .NET Framework. Dichos proveedores describen una colección de clases que se utiliza para obtener acceso a un origen de datos, como una base de datos, en el espacio administrado.
- **System.Data.OleDb:** clases que componen el proveedor de datos de .NET Framework para orígenes de datos compatibles con OLE DB. Estas clases permiten conectarse a un origen de datos OLE DB.
- **System.Data.SqlClient:** clases que conforman el proveedor de datos de .NET Framework para SQL Server, que permite conectarse a un origen de datos SQL Server 7.0.
- **System.Data.SqlTypes:** proporciona clases para tipos de datos nativos de SQL Server. Estas clases ofrecen una alternativa más segura y más rápida a otros tipos de datos.
- **System.Data.OracleClient:** clases que componen el proveedor de datos de .NET Framework para Oracle. Estas clases permiten el acceso a orígenes de datos Oracle en el espacio administrado.

51

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET



52

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. ADO.NET

- **Elemento fundamentales de ADO.NET**

ADO.NET utiliza algunos objetos ADO, como **Connection** y **Command**, y también agrega objetos nuevos. Algunos de los nuevos objetos clave de ADO.NET son **DataSet**, **DataReader** y **DataAdapter**.

- Objetos **Connection**. Para conectar con una base de datos y administrar las transacciones en una base de datos.
- Objetos **Command**. Para emitir comandos SQL a una base de datos.
- Objetos **DataReader**. Proporcionan una forma de leer una secuencia de registros de datos sólo hacia delante desde un origen de datos SQL Server.
- Objetos **DataSet**. Para almacenar datos sin formato, datos XML y datos relacionales, así como para configurar el acceso remoto y programar sobre datos de este tipo.
- Objetos **DataAdapter**. Para insertar datos en un objeto **DataSet** y reconciliar datos de la base de datos. .

53

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. .NET vs Otros

- **Entornos de Desarrollo. Caso de Estudio Tecnología .NET (~ 8 horas)**

- Características generales de .NET
- Ensamblados (Assemblies)
- Administración de datos con ADO.NET
- **.NET frente a otras tecnologías**
- El entorno Visual Studio .NET
- Lenguaje de Programación C#

54

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs COM**

- .NET supera ampliamente al modelo COM, es mucho más simple que éste y consigue eliminar las deficiencias que presentaba.
- Uno de los problemas actuales de COM es la limitación de su sistema de información sobre los tipos. En el modelo COM, la correspondencia entre el sistema de tipos de éste y el de alguno de los lenguajes que lo implementan, como C++, es en ocasiones complejo.
- En COM no existe una forma estándar de representar la información de tipos, pudiendo ser representado en formato de texto mediante las interfaces IDL o en forma binaria mediante las librerías de tipos, el problema se agrava aún más si tenemos en cuenta que hay casos en los que la información representada en un formato no tiene su equivalente en el otro.
- El sistema de información de tipos de COM tampoco mantiene la información sobre las dependencias de componentes externos, de forma que el sistema no puede determinar qué componentes deben estar instalados para que otro componente funcione correctamente.

55

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs COM**

- Otra limitación viene dada por el hecho de que únicamente son descritos los tipos expuestos por el componente, y no es posible disponer de información sobre los tipos internos, lo que hace que ciertos servicios sean imposibles, tales como la serialización automática.
- El sistema de información sobre los tipos de .NET, basado en el uso de los metadatos, mejora notablemente al sistema del modelo COM.
- Mediante los metadatos se consigue tener un único sistema de información sobre los tipos, manteniendo además la información relativa a las dependencias de un componente.
- .NET elimina el denominado “infierno de las DLLs” que va asociado al modelo COM. Dicho término se refiere a los problemas relativos al manejo de versiones que surgen cuando una DLL o un componente COM es compartido por varias aplicaciones.

56

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs COM**

- Otro de los problemas de COM es la complejidad asociada al despliegue e instalación de los componentes. El actual sistema de instalación de COM consta de varias etapas, copiándose una serie de componentes al disco y realizando una serie de entradas en el registro que describe a estos componentes en el sistema.
- Asimismo, la relación entre la imagen binaria y sus entradas en el registro es extremadamente frágil y débil. Frecuentemente se incluyen en el registro entradas para las coclases, interfaces, librerías de tipos, identificadores para DCOM, etc., por lo que es habitual tener que acabar manteniendo la consistencia de dichas entradas manualmente.
- Estos problemas, que derivan principalmente de la separación entre la descripción del componente y el componente en sí mismo, han intentado solucionarse en la plataforma .NET mediante la utilización de unidades de despliegue autodescriptivas, los ensamblados, eliminándose cualquier dependencia con el registro, lo que permite, en ocasiones, tener un proceso de despliegue tan simple como la copia de la aplicación al disco.

57

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs COM+**

- COM+ surgió como una mejora del Servidor de Transacciones de Microsoft MTS (*Microsoft Transaction Server*), mediante el cual era posible añadir atributos a un componente COM, de forma que se aplicasen sobre él una serie de servicios como transacciones, activación JIT, etc. asegurando los servicios requeridos.
- COM+ heredó muchas de las características de MTS, a la vez que amplió los servicios proporcionados por éste y mejoró la interoperabilidad con el modelo COM.
- Al contrario de lo que sucede con COM, no se puede decir que la plataforma .NET vaya a reemplazar, al menos por el momento, a COM+ como sistema proveedor de servicios de componentes, pues .NET carece de dichos servicios por sí mismo, teniendo que recurrir a la interoperabilidad con COM+ para poder ofrecer una serie de servicios a sus componentes.
- Este es uno de los aspectos en los cuales la plataforma .NET denota cierta inmadurez, pues no ofrece por sí sola una serie de servicios presentes en los modelos de componentes actuales como J2EE y CORBA, teniendo que recurrir a un modelo anterior.

58

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs COM+**

- La utilización de los servicios de COM+ esta bien integrada con el framework desde el punto de vista de la programación, pues el acceso a los mismos se realiza heredando de una clase base y mediante la utilización de atributos .NET.
- Sin embargo, desde el punto de vista de despliegue no puede decirse en absoluto lo mismo, pues éste choca frontalmente con los conceptos incluidos en .NET. Llama fuertemente la atención el hecho de que para acceder a los servicios de COM+ haya que volver a algo del anterior modelo en apariencia superado por .NET: el registro de una aplicación COM+, en el catálogo de éste, que hospeda a un componente que hace uso de los servicios de COM+.
- Se intenta disimular este hecho proporcionando un registro dinámico del componente (siempre que el usuario que ejecute la aplicación que contiene al componente pertenezca al grupo de usuarios administradores del sistema), y herramientas que simplifican enormemente dicho proceso de registro.

59

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs J2EE**

- Las plataformas .NET y J2EE son muy similares entre sí, manteniendo bastantes puntos en común. Simplificando mucho el análisis entre ambas se podría llegar a la conclusión de que:
 - J2EE es una plataforma que utiliza únicamente el lenguaje Java, multiplataforma y con múltiples proveedores de tecnología
 - .NET es multilenguaje diseñada para una única plataforma y con un único proveedor, Microsoft.
- Ambas plataformas poseen un modelo de componentes relativamente similar. Así, en J2EE el modelo de componentes viene dado por los JavaBeans y Enterprise JavaBeans, los cuales poseen los conceptos de propiedades, eventos que en el .NET aparecen incluidos en el sistema de tipos común, y por lo tanto, son soportados por los distintos lenguajes.
- Ambos modelos también poseen sistemas de acceso remotos (RMI en J2EE y el framework remoto en .NET), siendo el framework de .NET algo mas potente que RMI, además de poseer una capacidad de extensibilidad mayor.

60

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs J2EE**

- Soporte para el desarrollo de clientes delgados (*skinny clients*): J2EE posee a los servlets y a las páginas JSP, mientras que .NET posee a ASP.NET. Con ASP.NET es posible utilizar cualquier lenguaje y se introduce un modelo de desarrollo basado en componentes que permite abstraerse del navegador cliente.
- Para el desarrollo de los clientes pesados (*fat clients*) ambas plataformas poseen sus librerías y su colección de controles. Para J2EE esta Java Swing, y para .NET están los WinForms, los cuales son bastante similares, diferenciándose en determinadas cuestiones tales como la gestión de los eventos, que con las Java Swing se realiza mediante clases internas y en .NET se realiza mediante el sistema de eventos basado en delegados.
- Ambas plataformas poseen APIs para el acceso a fuentes de datos, JDBC para J2EE y ADO.NET para .NET. En este aspecto también es superior .NET, pues introduce la posibilidad de trabajar en modo desconectado con los denominados DataSet y está altamente integrado con XML, realizándose la transmisión de datos entre una fuente de datos y un DataSet en XML, lo que permite tratar a cualquier cosa que emita XML con el esquema de ADO.NET como una fuente de datos.

61

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs J2EE**

- En lo relativo al despliegue, ambos modelos utilizan una noción similar de empaquetado, denominándose a dichas unidades en J2EE módulos. En ambas plataformas los empaquetados contienen un manifiesto en el cual se expresan sus dependencias externas así como otra serie de metainformación.
- La mayor diferencia viene dada por el runtime de ambas plataformas. La máquina virtual de Java ha sido diseñada para proporcionar soporte multiplataforma al lenguaje Java, mientras que el CLR de .NET ha sido diseñado también con el objetivo de ser independiente de la plataforma pero también con el de serlo del lenguaje.
- Con respecto a la comparación entre los bytecodes de Java y el MSIL de .NET, MSIL es de mayor nivel que los bytecodes e introduce la seguridad de tipos, pudiendo ser verificado el código MSIL antes de su ejecución para garantizar su seguridad.
- En la ejecución del código intermedio también se siguen esquemas distintos; así, en Java los bytecodes son interpretados, mientras que el CLR compila el código MSIL a código nativo, obteniendo un mayor rendimiento la plataforma .NET.

62

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs J2EE**

- Un punto donde .NET es superado ampliamente por J2EE son los servicios ofrecidos a los componentes tales como persistencia automática, transacciones, etc.
- Además, el hecho de que .NET no tenga servicios propios y los tome de COM+ constituye un factor en contra de .NET.
- Por último, si establecemos una comparación entre las herramientas de desarrollo existentes entre ambas plataformas podemos llegar a la conclusión de que en J2EE hay una amplia gama de herramientas de distintos proveedores, sin embargo éstas carecen de la integración que posee VisualStudio.NET, lo que es un punto a favor de este último, a pesar de que las herramientas de J2EE son tan potentes o más que VisualStudio.

63

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs CORBA**

- La plataforma .NET es más amplia que la especificación CORBA en el sentido de que cubre una serie de aspectos no tratada por ésta, como puede ser la parte de las interfaces de usuario (ASP.NET y WinForms). .NET llega mucho más lejos que CORBA en lo que se refiere a la integración entre distintos lenguajes, pues en .NET una clase de otro lenguaje es considerada como una clase del propio, pudiendo heredar entre sí clases en distintos lenguajes.
- Asimismo, CORBA carece de portabilidad en la implementación, a no ser, claro está, que se utilice Java.
- .NET dispone también de un framework de objetos remotos, sin embargo, es evidente que el framework CORBA supera ampliamente al de .NET. A pesar de que el framework de .NET posee una alta extensibilidad, permitiendo utilizar cualquier protocolo y formato de serialización, CORBA es mucho más potente y mucho más flexible, sobre todo en lo relativo a la configuración de las políticas de configuración de los objetos remotos.

64

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. NET vs Otros

- **.NET vs CORBA**

- Además de los objetos remotos, .NET introduce los servicios Web para el desarrollo de aplicaciones distribuidas, si bien hay que reconocer que desde un punto de vista técnico los servicios Web son poco más que un RPC basado en la utilización de protocolos estándares (HTTP, XML, etc.), éstos, a diferencia de otros modelos como CORBA y DCOM, son independientes del modelo de objetos, incluidos el del propio .NET, lo que les dota de una alta interoperabilidad.
- Los servicios Web permiten un desarrollo de aplicaciones distribuidas mucho más ligero que el impuesto por CORBA o por los objetos remotos de .NET, pues bastaría con tener una librería de SOAP, o en su defecto, un parser XML, para acceder a un servicio Web.
- A favor de CORBA está el hecho de que se trata de una especificación estándar consensuada por las distintas organizaciones y empresas que forman el OMG.
- Posiblemente, una de las cuestiones por las que CORBA no ha alcanzado la difusión que se merecía es la ausencia de entornos de desarrollo tales como el entorno VisualStudio que posee la plataforma .NET.

Ramón Hervás Lucas - Curso 2007/2008 - HyEP

65

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Entornos de Desarrollo. Caso de Estudio Tecnología .NET (~ 8 horas)**

- Características generales de .NET
- Ensamblados (Assemblies)
- Administración de datos con ADO.NET
- .NET frente a otras tecnologías
- **El entorno Visual Studio .NET**
- Lenguaje de Programación C#

66

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Características generales de VS.NET**
 - Entorno integrado de desarrollo
 - Independiente del lenguaje de desarrollo.
 - Permite desarrollar soluciones combinando varios lenguajes y compartiendo un único interfaz.
 - Depuración extremo a extremo de las aplicaciones independientemente del número de proyectos, procesos y procedimientos
 - Ayuda dinámica (Dynamic Help)
 - Editor visual de páginas Web (Visual Web Page Editor)
 - Lista de tareas (Task List)
 - Examinador de Objetos (Object Browser)
 - Nueva funcionalidad de ventana de comandos (Command Windows)
 - Ventanas de ocultación automática (Auto-Hide)

67

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Características generales de VS.NET**
 - **Ayuda dinámica**
 - El objetivo es encontrar la información correcta en el momento oportuno.
 - Ofrece documentación relacionada en función de las características y tecnologías en uso
 - Ejemplo
 - Si se está utilizando el IDE pero no se ha abierto ninguna aplicación o componente, el entorno ofrece enlaces a información relacionada con la forma de planear un aplicación, selección de plantillas de negocio así como otras plantillas dinámicas de diferentes proveedores. Conforme se avanza en el desarrollo de la aplicación, el IDE analiza la parte de la aplicación en la que se está trabajando y muestra contenidos apropiados.

68

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Características generales de VS.NET**

- **Editor visual de páginas Web**

- Editor compartido de tipo WYSIWYG
- Permite desarrollar páginas Web sin necesidad de entrar a fondo en el código HTML o de secuencia de comandos.
- Facilidades:
 - Compleción de sentencias y etiquetas HTML
 - Comprobación de sintaxis XML durante la fase de diseño
 - Posicionamiento absoluto de elementos

69

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Características generales de VS.NET**

- **Lista de tareas**

- Permite marcar el código con comentarios relacionados con las tareas que se necesitan realizar.
- Estas tareas se analizan sintácticamente y se muestran en un formato tabular.
- Esta característica hace que resulte sencillo anotar el código de forma que, cuando uno mismo, o cualquier otro miembro del equipo, sea posible conocer el estado exacto del código con un mínimo esfuerzo

- **Explorador de Objetos**

- Conecta todos los objetos del sistema y proporciona información detallada acerca de cada uno de ellos.
- Es posible buscar información que se necesite a través de filtrado, agrupación y ordenación con independencia del lenguaje

70

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Características generales de VS.NET**

- **Ventana de Comandos**

- Permite sacar provecho con rapidez al proporcionar una única línea de acceso para buscar, explorar y ejecutar los numerosos elementos posibles, dentro y fuera del IDE.
 - Realizar búsquedas
 - Explorar ventanas y elementos de la solución
 - Ejecutar comandos
 - Explorar el sitio web
 - Ejecutar programas externos

71

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **Características generales de VS.NET**

- **Ocultación automática**

- Cuando se deja de utilizar una ventana, una barra de herramientas, se comprime a un lado para optimizar la visión del IDE
- De esta forma se permite un mayor espacio para el desarrollo de código o el diseño de interfaces
- Otra forma de optimizar el espacio es la utilización de varios monitores. VS.NET facilita la distribución de componentes del IDE en varios monitores.

72

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Características

- **VS.NET para desarrollo empresarial**

- La tecnología .NET proporciona a las organizaciones la posibilidad de definir directivas y recomendaciones acerca de los proyectos, comunicarlasy así reforzar las decisiones tecnológicas y de arquitectura.
- Componentes para desarrollo empresarial:
 - Plantillas empresariales: permite la creación de plantillas para soluciones habituales.
 - Definición de Directivas: permitiendo el filtrado de opciones de menú, de diálogo y de componentes

73

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Novedades principales de VS.NET 2005**

- **Soporte para refactoring:** Refactorizar es el proceso de modificar el código fuente pero sin modificar su temática. El objetivo hacer el código fuente más intangible y menos propenso a errores. El soporte para refactorización de Visual Studio 2005 permite, entre otros, convertir variables públicas a propiedades, promover variables locales a parámetros, etc.
- **Herramientas de testing integradas;** Permite realizar tests unitarios, tests de estrés de aplicaciones ASP.NET y tests de cobertura.
- Nueva herramienta de **control de código fuente**. Proporciona una experiencia más orientada a la colaboración, con herramientas de fusión manual y automático para soportar mejor operaciones de *múltiples usuarios*.
- **Just my code debugging.** El depurador se puede configurar para que ignore código fuente de *terceros* y depure sólo nuestro código fuente.

74

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Novedades principales de VS.NET 2005**

- **Smart tags:** La funcionalidad más importante de controles (wincontrols y webcontrols) se agrupa en *smart tags*, Haciendo clic sobre el smart tag de un control se nos da acceso a un menú que se nos permite de forma fácil realizar las acciones más comunes sobre aquel control.
- **Edit-and-continue:** Podemos depurar, cambiar código mientras estamos depurando (en un punto de ruptura) y en algunos casos, seguir la depuración.
- **Auto using:** El entorno es capaz incluir automáticamente aquellas sentencias “using” que no hayamos puesto en el código fuente y que sean necesarias para las clases que estemos utilizando.
- **Soporte para SQL Server 2005:** posibilidad de desarrollar componente para SQL Server 2005, directamente desde Visual Studio 2005.

75

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Novedades principales de ASP.NET**

- **Master pages:** En ASP.NET 1.x si se quiere mantener un *layout* consistente en un conjunto de páginas, era necesario encapsular este layout en uno o varios controles web (.ascx) e incrustar los controles en cada página. En ASP .NET 2.0 se propone seguir el enfoque contrario: el layout común es diseñar una sola vez y se coloca en un único sitio: la página principal (*master page*). A cada página principal se define uno o más “sostenedores” que en tiempo de ejecución contendrán las diferentes páginas.
- **Expression syntax:** Permite definir expresiones que se evaluarán antes de procesar la página. Se utiliza en el nuevo sistema de localización implementado en ASP.NET y se puede usar también para establecer determinados valores de control de forma dinámica, antes de que la página ASP.NET sea procesada por el Framework (p. ej. Cadenas de conexión)
- **Nuevos controles de alto nivel:** Controles de login/logout, mapas de sitios web, *wizards*, etc.

76

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Novedades principales de ASP.NET**

- **Herramienta de configuración:** Una nueva herramienta de configuración (web) permite configurar el fichero web config sin necesidad de editar el XML.
- **Localización basada en ficheros de recursos:** Similar a los Winforms de Visual Studio .NET 2003: Un fichero de recursos para cada sitio web y en tiempo de ejecución el motor de ASP.NET substituirá automáticamente los valores de las propiedades de los controles indicador para los valores contenidos en el fichero de recursos utilizados.
- **API para personalización y para *webparts*.**
- **Nuevo servidor web integrado.** No es necesario tener un IIS instalado en la máquina para crear y depurar aplicaciones ASP.NET. Adicionalmente las aplicaciones ASP.NET pueden crearse en cualquier directorio de disco (sin necesidad de definir ningún directorio virtual).
- **Nuevo API de configuración más flexible.**

77

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Novedades principales de ADO.NET**

- Permiten abstraer el resto de las aplicaciones del proveedor de datos utilizado. No es propiamente una novedad de ADO.NET sino de ASP.NET y de WinForms.
- Objeto DataTableReader permite iterar para los valores de un DataTable de forma rápida y eficiente.
- Posibilidad de cambiar manualmente el RowState de una o varias filas de un DataTable.

- **Novedades principales de Compact Framework**

- SQL Server 2005 Mobile que substituye SQL Server CE y ofrece un rendimiento superior y la posibilidad de tener más de una conexión simultánea contra una misma BBDD. Además las BBDD de SQL Server 2005 Mobile pueden gestionarse a través de Visual Studio 2005 (o del SQL Management Studio).
- Soporte para clases del .NET Framework que no estaban soportadas en la versión anterior (a destacar entre otros; soporte para el registro, MSMQ, hilos y puertos serie).

78

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Entorno integrado de desarrollo compartido**
 - Una de las principales características de la tecnología .NET es la integración de su IDE único para todos los lenguajes que soporta.
 - Página de Inicio
 - Explorador de Soluciones
 - Cuadro de herramientas
 - Explorador de Servidores
 - Lista de Tareas
 - Ayuda dinámica
 - Ventanas de Documentos
 - Ventana de Comandos
 - Otras funciones

79

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Página de Inicio**
 - Cada vez que VS.NET se arranca se muestra la página de inicio.
 - Ofrece opciones para abrir o crear nuevos proyectos y sitios web, contenidos de ayuda básicos y artículos de MSDN relacionados con nuestro perfil.



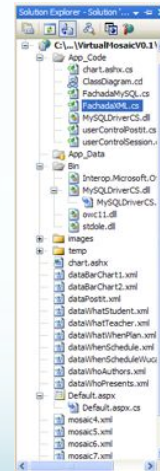
80

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Explorador de Soluciones**

- Muestra una lista organizada de proyectos así como los correspondientes archivos y directorios que forman parte de la solución actual.
- Algunas de las funciones que podemos realizar desde el explorador de soluciones son:
 - Añadir archivos nuevos o existentes
 - Asociar referencias de componentes externos (DLL, COM+, etc)
 - Crear y visualizar el diagrama de clases de la solución.
 - Opciones generales de configuración de la solución



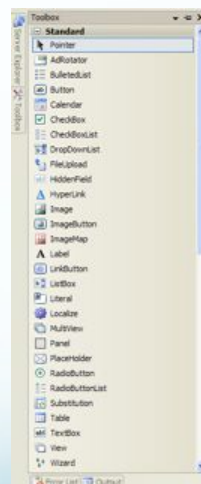
81

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Cuadro de herramientas**

- Muestra los distintos elementos que se pueden utilizar en los proyectos de Visual Studio. Los elementos disponibles varían según el contexto.
- Algunos de los elementos que pueden aparecer son:
 - Controles de formularios basado en Web o en Aplicaciones de Ventana
 - Controles de ActiveX
 - Servicios Web
 - Elementos de lenguaje de marcas hipertextuales (HTML)
 - Objetos y elementos del portapapeles.



82

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Explorador de Servidores**

- Consola de desarrollo de servidores para Visual Studio .NET
- Ayuda a acceder y manipular recursos de cualquier equipo en el que se tenga permisos.
- Permite realizar conexiones con servidores para ver sus recursos, incluyendo colas de mensajes, contadores de rendimiento, servicios, procesos, registro de sucesos, servicios Web y objetos de bases de datos.
- Además permite hacer referencia de forma remota a componentes y recursos .NET



83

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Lista de tareas**

- Además de escribir código y de crear componentes, el desarrollador debe ser capaz de anotar su código de forma que, cuando el mismo u otros miembros del equipo lo examinen puedan determinar el estado exacto del código sin demora.
- La lista de tareas permite marcar el código con comentarios.
- La lista también sirve como ubicación central donde determinar el estado de los errores e identificar y ubicar los problemas

- **Ayuda dinámica**

- Proporciona acceso con un solo *clic* a la ayuda adecuada.
- Para ello se realiza un seguimiento de las selecciones que realiza el desarrollador, la ubicación del cursor y los elementos activos del propio IDE.

84

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Ventanas de documentos**

- **IntelliSense:** Aplicado a los lenguajes compilados pero también a lenguajes de marcas como HTML o XML obteniendo información inmediata sobre las etiquetas disponibles
- **Depurador integrado:** Facilita la ejecución, traza y corrección de errores. A través de puntos de interrupción podemos navegar por el estado completo de la solución, incluyendo componentes distribuidos o remotos. Además el depurador de .NET tiene la facultad de agregarse a un programa que se ejecute fuera de Visual Studio.

- **Ventana de Comandos:** Mecanismo flexible para la ejecución de comandos sin necesidad de acceder al menú y saltándonos cuadros de diálogo.

85

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Diseñadores**

- Diseñador de formularios web: Facilita una forma intuitiva de crear páginas Web sin tener que adentrarse en código HTML.
- Diseñador de formularios de Windows: Proporciona un conjunto de clases extensible para la construcción rápida de herramientas basadas en Windows. Permite la integración total de controles externos.
- Diseñador de Componentes: Para el desarrollo rápido de componentes del lado el servidor.
- Diseñador XML: Proporciona herramientas intuitivas para trabajar con XML y archivos de definición de esquemas XSD. Dentro del diseñador existen tres vistas: una para crear y editar esquemas XSD, otra para la edición estructurada de archivos de datos XML y la última para edición de código XML. Otra funcionalidad interesante es la posibilidad de generar conjuntos de datos de ADO.NET

- **Macros**

- Visual Studio posee un modelo enriquecido para la personalización y automatización del entorno de desarrollo.

86

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2005

- **Administración de datos en aplicaciones Web**
 - ADO+ es una mejora de ADO que proporciona interoperatividad de datos entre plataformas.
 - El receptor de los datos no tiene porque ser un objeto ADO+ gracias a las características de XML
 - Los conjuntos de datos son nuevos en ADO+: Un conjunto de datos (*dataset*) es una copia en memoria de datos de una base de datos. Típicamente se corresponde a una tabla o a una vista de una tabla.
 - En tiempo de ejecución, los datos se pasan desde la base de datos a un objeto ADO que lo convierte en XML.
 - El receptor de los datos los lee como si de un fichero XML en memoria se tratase, abstrayéndose de la red y de la propia transmisión de datos.
 - La programación actúa sobre objetos de dato en vez de sobre tablas y columnas
 - If TotalCost > Table("Cliente").Column("CreditoDisponible")
 - If TotalCost > Cliente.CreditoDisponible

87

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2008

- **Visual Studio 2008 proporciona avances para los desarrolladores en tres aspectos principales:**
 - Aumentar la productividad del desarrollador
 - Soporte a todo el ciclo de vida
 - Soporte para las tecnologías más recientes
- **Aplicados a 7 áreas fundamentales**
 - Desarrollo de aplicaciones cliente
 - Desarrollo de aplicaciones basadas en Microsoft Office
 - Desarrollo de aplicaciones para Windows Vista
 - Manejo de datos de forma más productiva
 - Aumento de la experiencia global del desarrollador
 - Experiencias Web (AJAX)
 - Gestión del ciclo de vida (ALM)

88

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2008

- **Desarrollo de aplicaciones cliente**
 - **Diseño integrado de Interfaces de Usuario**, mediante XAML y otorgando mayor control sobre los elementos y la forma en que se muestran.
 - **Soporte para aplicaciones “ClickOnce”**, descargables e instalables a través de una URL.
 - **Soporte para desarrollar aplicaciones que utilicen el estilo Office 2007.**
 - **Acceso a datos ocasional** (como tercer modelo junto al acceso sincronizado y no sincronizado)
 - **Integración con SQL-Server Compact Edition**

89

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2008

- **Desarrollo de aplicaciones basadas en Microsoft Office**
 - Integración de *Visual Studio for Office (VSTO)*
 - Construir aplicaciones sobre un servidor *SharePoint Workflow*
 - Fácil integración de controles propios de *Office*
- **Desarrollo de aplicaciones para Windows Vista**
 - Herramientas para *Windows Presentation Foundation*
 - Interoperatividad entre código nativo y código supervisado
 - Se incluyen 8000 librerías específicas de *Windows Vista*

90

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2008

- **Manejo de datos de forma más productiva**
 - Integración de ADO.NET y LINQ (*Language Integrated Query*) para unificar el acceso a datos y abstraer de la naturaleza de estos.
 - Librerías para acceso a objetos de datos (bases de datos, XML, etc.) sin necesidad de serializar consultas (*Xpath* o SQL)
- **Aumento de la experiencia global del desarrollador**
 - Unificación de plataformas
 - Integración de *WindowsForm* y los componentes desarrollados

91

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. VS.NET 2008

- **Experiencias Web**
 - Proporciona soporte para programación Web estilo-AJAX
 - Mayor soporte para diseño e implementación de Servicios Web
 - Servicios de *Windows Communication Foundation*, facilitando la creación de clientes y *proxies*.
- **Gestión del ciclo de vida (ALM)**
 - Nuevas herramientas para refactorización, generación de casos de prueba, diagnósticos, etc.

92

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Entornos de Desarrollo. Caso de Estudio Tecnología .NET (~ 8 horas)**

- Características generales de .NET
- Ensamblados (Assemblies)
- Administración de datos con ADO.NET
- .NET frente a otras tecnologías
- El entorno Visual Studio .NET
- **Lenguaje de Programación C#**

93

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- Durante las décadas de los 80 y 90, C y C++ han sido los lenguajes más utilizados para el desarrollo de software comercial y de negocio.
- Son lenguajes que proporcionan un gran control sobre los pequeños detalles pero esta flexibilidad supone un coste en productividad (Una misma aplicación en C o C++ suele tardar más en desarrollarse que en Visual Basic)
- El objetivo es tener un lenguaje que equilibre la potencia y la productividad
- La solución ideal sería un desarrollo rápido combinado con la potencia de acceso a toda la funcionalidad de la plataforma subyacente. Además el entorno debe estar en sintonía con los estándares Web actuales y que proporcionen una integración sencilla con las aplicaciones existentes.
- Por último es deseable mantener la posibilidad de codificar a bajo nivel pero sólo cuando sea imprescindible.

94

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **El lenguaje C#**

- Lenguaje sencillo, con protección de tipos y orientado a objetos
- Lenguaje para RAD (Rapid Application Development)
- Instrucciones, expresiones y operadores similares a C++ (99%) y Java.
- Interoperatividad completa con COM y COM+
- Recolección automática de basura
- Seguridad de tipos: no existen variables no inicializadas ni conversiones no seguras, además, se comprueba el rango de acceso a los *arrays* y el desbordamiento de operaciones
- Metadatos extensibles y tipados, los que permite declaración de nuevos tipos.
- Soporte XML para interacción de componentes Web

95

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- **Productividad y seguridad**

- Actualmente se presiona a los desarrolladores para que acorten los intervalos de tiempo y para producir muchas revisiones incrementales en lugar de una gran versión final.
- C# está diseñado precisamente para ayudar a conseguir más con menos líneas de código y menores posibilidades de error

- **Adopción de estándares de programación Web**

- Cada vez más soluciones requieren de utilización de estándares del Web (HTML, XML, SOAP, etc.)
- .NET incluye el soporte para convertir cualquier componente en un servicio Web invocable desde Internet. Para el desarrollador, un servicio Web tiene el mismo aspecto que un objeto.

96

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- **Eliminación de errores de programación**

- Incluso los programadores expertos cometen errores sencillos. Con frecuencia esos pequeños errores provocan problemas impredecibles que pueden permanecer ocultos durante largos periodos de tiempo
- Una vez que una solución esté en fase de producción resulta muy costoso reparar un error.
- El diseño de C# ayuda a la eliminación de errores:
 - La recolección de basura libera al desarrollador de la administración manual de la memoria
 - En C# el entorno inicializa automáticamente las variables
 - Las variables pertenecen a tipos seguros

- **Mantenimiento de versiones**

- Las revisiones en el código pueden modificar de forma no intencionada la semántica del programa existente.
- C# incluye un lenguaje de mantenimiento de versiones

97

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- **Correspondencia entre procesos de negocio y su implementación**

- Resulta imperativo disponer de una conexión muy estrecha entre los procesos de negocio abstractos y la implementación de software real.
- C# admite metadatos tipados y extensibles que se apliquen a cualquier objeto. Así la comunicación entre una arquitecto de proyecto y el desarrollador es más completa.

- **Interoperatividad extensiva**

- En C# cada objeto es, automáticamente un objeto COM sin necesidad de implementar una interfaz de componente.

- **Escalabilidad**

- C# no impone ninguna restricción acerca de dónde están ubicados los archivos fuente o cómo se llaman. Cuando se compila un proyecto C# se puede considerar que es como una concatenación de todos los archivos fuente.

98

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- **Simplicidad**

- Reducción de operadores: Desaparecen los operadores de puntero (->), miembros de clase (::) para utilizar el punto (.) como operador único para miembros de clases, espacios de nombres, referencias, etc.
- Tipos de datos: se simplifican y se abstrae su representación final. Son tipos independientes del compilador, sistema operativo o máquina. Ejemplo: char en vez de signed char, unsigned char y wchar_t
- Tipo bool: En C típicamente se utilizan enteros como booleanos.
- Mejora de la función switch



```
switch (i){  
    case1: FunctionA();  
    case2: FunctionB();  
    Break;  
}
```

```
switch (i){  
    case1: FunctionA();  
    goto case 2;  
    case2: FunctionB();  
    Break  
}
```



99

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- **Consistencia**

- Unificación del sistema de tipos permitiendo ver todos los tipos del lenguaje como objetos.
- Cuando se utiliza una clase, una estructura, un array o una primitiva siempre se les puede tratar como objetos
 - Ejemplo: `System.Console.WriteLine ("Hello World");`
System es un espacio de nombres, *Console* una clase y *WriteLine* un método.
- Los espacios de nombres permiten la consistencia ante nombres repetidos o distintas versiones.

- **Modernidad**

- Entre otros aspectos como la recolección de basura y el soporte integral para errores, C# proporciona directivas para controlar el flujo del programa.
 - Ejemplo: `[Conditional("DEBUG")]`

100

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- **Orientación a objetos**

- “He conocido personalmente a gente que ha trabajado con herencia múltiple durante una semana para finalmente mudarse, totalmente frustrados, a Carolina del Norte y dedicarse a cuidar cerdos”

Joshua Trupin, editor técnico de MSDN Magazine

- C# entierra la herencia múltiple a favor del soporte nativo de objetos virtuales COM+
 - C# entierra también el concepto de funciones, variables y constantes globales. En su lugar se pueden crear miembros de clases estáticas.
 - La sobrecarga de métodos es más sencilla y sólo es posible en el ámbito local

```
Interface Itest{  
    void F ( );  
    void F (int x);  
    void F (string x);  
}
```

Ramón Hervás Lucas - Curso 2007/2008 - HyEP

101

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- **Orientación a componentes**

- La propia sintaxis de C# incluye elementos propios del diseño de componentes que otros lenguajes tienen que simular mediante construcciones más o menos complejas. Es decir, la sintaxis de C# permite definir cómodamente **propiedades** (similares a campos de acceso controlado), **eventos** (asociación controlada de funciones de respuesta a notificaciones) o **atributos** (información sobre un tipo o sus miembros)

- **Gestión automática de memoria**

- todo lenguaje de .NET tiene a su disposición el recolector de basura del CLR.
 - Esto tiene el efecto en el lenguaje de que no es necesario incluir instrucciones de destrucción de objetos.
 - Sin embargo, dado que la destrucción de los objetos a través del recolector de basura no es determinista y sólo se realiza cuando éste se active -ya sea por falta de memoria, finalización de la aplicación o solicitud explícita en el fuente-, C# también proporciona un mecanismo de liberación de recursos determinista a través de la instrucción **using**

Ramón Hervás Lucas - Curso 2007/2008 - HyEP

102

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Introducción a C#**

- **Sistema de tipos unificado**

- A diferencia de C++, en C# todos los tipos de datos que se definan siempre derivarán, aunque sea de manera implícita, de una clase base común llamada **System.Object**, por lo que dispondrán de todos los miembros definidos en ésta clase (es decir, serán "objetos")
 - A diferencia de Java, en C# esto también es aplicable a los tipos de datos básicos. Además, para conseguir que ello no tenga una repercusión negativa en su nivel de rendimiento, se ha incluido un mecanismo transparente de **boxing** y **unboxing** con el que se consigue que sólo sean tratados como objetos cuando la situación lo requiera, y mientras tanto puede aplicárseles optimizaciones específicas.
 - El hecho de que todos los tipos del lenguaje deriven de una clase común facilita enormemente el diseño de colecciones genéricas que puedan almacenar objetos de cualquier tipo

103

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **El Preprocesador de C#**

- El **preprocesado** es un paso previo a la compilación mediante el que es posible controlar la forma en que se realizará ésta.
 - El **preprocesador** es el módulo auxiliar que utiliza el compilador para realizar estas tareas, y lo que finalmente el compilador compila es el resultado de aplicar el preprocesador al fichero de texto fuente, resultado que también es un fichero de texto.
 - Mientras que el compilador hace una traducción de texto a binario, lo que el preprocesador hace es una traducción de texto a texto.
 - En C# se han eliminado la mayoría de características de C++ que provocaban errores difíciles de detectar (macros, directivas de inclusión, etc.) y prácticamente sólo se usa para permitir realizar compilaciones condicionales de código.
 - El nombre que se indica tras el símbolo # es el nombre de la directiva, y el texto que se incluye tras él (no todas las directivas tienen por qué incluirlo) es el valor que se le da. Por tanto, la sintaxis de una directiva es:

`#<nombreDirectiva> <valorDirectiva>`

104

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **El Preprocesador de C#**

- La principal utilidad del preprocesador en C# es la de permitir determinar cuáles regiones de código de un fichero fuente se han de compilar. Para ello, lo que se hace es encerrar las secciones de código opcionales dentro de directivas de compilación condicional, de modo que sólo se compilarán si determinados identificadores de preprocesado están definidos.
- Es importante señalar que cualquier definición de identificador ha de preceder a cualquier aparición de código en el fichero fuente. Por ejemplo, el siguiente código no es válido, pues antes del **#define** se ha incluido código fuente

```
class A
#define PRUEBA
{ }
```

- Del mismo modo que es posible definir identificadores de preprocesado, también es posible eliminar definiciones de este tipo de identificadores previamente realizadas. Para ello la directiva que se usa tiene la siguiente sintaxis:

```
#undef <nombreIdentificador>
```

105

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **El Preprocesador de C#**

- **Compilación condicional**

- Para conseguir esto se utiliza el siguiente juego de directivas:

```
#if <condición1>
    <código1>
#elif <condición2> <código2>
    ...
#else
    <códigoElse>
#endif
```

- Como se ve en el ejemplo, las condiciones especificadas son nombres de identificadores de preprocesado, considerándose que cada condición sólo se cumple si el identificador que se indica en ella está definido. O lo que es lo mismo: un identificador de preprocesado vale cierto (**true** en C#) si está definido y falso (**false** en C#) si no.

106

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- El Preprocesador de C#

- Generador de avisos y errores

- El preprocesador de C# también ofrece directivas que permiten generar avisos y errores durante el proceso de preprocesado en caso de que ser interpretadas por el preprocesador. Estas directivas tienen la siguiente sintaxis:

```
#warning <mensajeAviso>
```

```
#error <mensajeError>
```

- La directiva **#warning** lo que hace al ser procesada es provocar que el compilador produzca un mensaje de aviso que siga el formato estándar usado por éste para ello y cuyo texto descriptivo tenga el contenido indicado en <mensajeAviso>; y **#error** hace lo mismo pero provocando un mensaje de error en vez de uno de aviso.

```
#warning Código aun no revisado
```

```
#define PRUEBA
```

```
#if PRUEBA && FINAL
```

```
#error Un código no puede ser simultáneamente de prueba y versión final
```

```
#endif
```

```
class A {}
```

107

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- El Preprocesador de C#

- Marcación de regiones de código

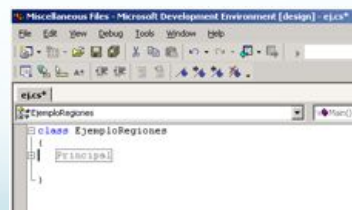
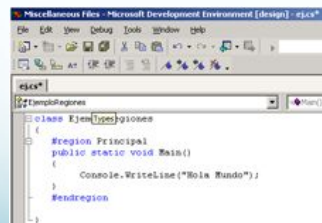
- Es posible marcar regiones de código y asociarles un nombre usando el juego de directivas **#region** **#endregion**. Estas directivas se usan así:

```
#region <nombreRegión>
```

```
<código>
```

```
#endregion
```

- La utilidad que se dé a estas marcaciones depende de cada herramienta. Visual Studio.NET, donde se usa para marcar código de modo que desde la ventana de código podamos expandirlo y contraerlo con una única pulsación de ratón.



108

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos léxicos

- Comentarios

- En C# hay dos formas de escribir comentarios. La primera consiste en encerrar todo el texto que se desee comentar entre caracteres `/*` y `*/` siguiendo la siguiente sintaxis:

- ```
/*<texto>*/
```

- Estos comentarios pueden abarcar tantas líneas como sea necesario. Por ejemplo:

- ```
/* Esto es un comentario que ejemplifica cómo se escribe comentarios que ocupen varias líneas */
```

- Ahora bien, hay que tener cuidado con el hecho de que no es posible anidar comentarios de este tipo. Es decir, no vale escribir comentarios como el siguiente:

- ```
/* Comentario contenedor /* Comentario contenido */ */
```

- También es posible incluir un comentario iniciando la línea con `//`. como indicador de su final el fin de línea

109

## Herramientas y Entornos de Programación

### Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos léxicos

- Identificadores

- Típicamente el nombre de un identificador será una secuencia de cualquier número de caracteres alfanuméricos -incluidas vocales acentuadas y eñes- tales que el primero de ellos no sea un número.
    - Sin embargo, y aunque por motivos de legibilidad del código no se recomienda, C# también permite incluir dentro de un identificador caracteres especiales imprimibles tales como símbolos de diéresis, subrayados, etc. siempre y cuando estos no tengan un significado especial dentro del lenguaje.
    - Finalmente, C# da la posibilidad de poder escribir identificadores que incluyan caracteres Unicode que no se puedan imprimir usando el teclado de la máquina del programador o que no sean directamente válidos debido a que tengan un significado especial en el lenguaje. Para ello, lo que permite es escribir estos caracteres usando **secuencias de escape**, que no son más que secuencias de caracteres con las sintaxis

- ```
C\u0064
```

 (equivale a C#)

110

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos léxicos

- Palabras reservadas

- Los siguientes nombres no son válidos como identificadores ya que tienen un significado especial en el lenguaje:

abstract, as, base, bool, break, byte, case, catch, char, checked, class, const, continue, decimal, default, delegate, do, double, else, enum, event, explicit, extern, false, finally, fixed, float, for, foreach, goto, if, implicit, in, int, interface, internal, lock, is, long, namespace, new, null, object, operator, out, override, params, private, protected, public, readonly, ref, return, sbyte, sealed, short, sizeof, stackalloc, static, string, struct, switch, this, throw, true, try, typeof, uint, ulong, unchecked, unsafe, ushort, using, virtual, void, while

- Aparte de estas palabras reservadas, si en futuras implementaciones del lenguaje se decidiese incluir nuevas palabras reservadas, Microsoft dice que dichas palabras habrían de incluir al menos dos símbolos de subrayado consecutivos (__)
 - Aunque directamente no podemos dar estos nombres a nuestros identificadores, C# proporciona un mecanismo para hacerlo indirectamente y de una forma mucho más legible que usando secuencias de escape. Este mecanismo consiste en usar el carácter @ para prefijar el nombre coincidente con el de una palabra reservada

111

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos léxicos

- Literales

- Un literal es la representación explícita de los valores que pueden tomar los tipos básicos del lenguaje.
 - Literales enteros: Un número entero se puede representar en C# tanto en formato decimal como hexadecimal. En el primer caso basta escribir los dígitos decimales (0-9) del número unos tras otros, mientras que en el segundo hay que preceder los dígitos hexadecimales (0-9, a-f, A-F) con el prefijo 0x.

Ejemplos de literales enteros son 0, 5, +15, -23, 0x1A, -0x1a, etc

- Literales reales: Los números reales se escriben de forma similar a los enteros, aunque sólo se pueden escribir en forma decimal y para separar la parte entera de la real usan el tradicional punto decimal (carácter .) También es posible representar los reales en formato científico, usándose para indicar el exponente los caracteres e ó E.

Ejemplos de literales reales son 0.0, 5.1, -5.1, +15.21, 3.02e10, 2.02e-2, 98.8E+1, etc.

- Literales lógicos: Los únicos literales lógicos válidos son true y false, que respectivamente representan los valores lógicos cierto y falso

112

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos léxicos

- Literales (continuación)

- **Literales de carácter:**

Prácticamente cualquier carácter se puede representar encerrándolo entre comillas simples. Las únicas excepciones a esto son los caracteres que se muestran en la tabla, que han de representarse con secuencias de escape que indiquen su valor como código Unicode o mediante un formato especial tal y como se indica a continuación:

Carácter	Código de escape Unicode	Código de escape especial
Comilla simple	\u0027	\'
Comilla doble	\u0022	\"
Carácter nulo	\u0000	\0
Alarma	\u0007	\a
Retroceso	\u0008	\b
Salto de página	\u000C	\f
Nueva línea	\u000A	\n
Retorno de carro	\u000D	\r
Tabulación horizontal	\u0009	\t
Tabulación vertical	\u000B	\v
Barra invertida	\u005C	\\

- **Literales de cadena:** Una cadena no es más que una secuencia de caracteres encerrados entre comillas dobles

113

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos Léxicos

- Operadores

- **Operaciones aritméticas:** Los operadores aritméticos incluidos en C# son los típicos de suma (+), resta (-), producto (*), división (/) y módulo (%). También se incluyen operadores de "menos unario" (-) y "más unario" (+).
- **Operaciones lógicas:** Se incluyen operadores que permiten realizar las operaciones lógicas típicas: "and" (&& y &), "or" (|| y |), "not" (!) y "xor" (^).
- **Operaciones relacionales:** Se han incluido los tradicionales operadores de igualdad (==), desigualdad (!=), "mayor que" (>), "menor que" (<), "mayor o igual que" (>=) y "menor o igual que" (<=).
- **Operaciones de manipulación de bits:** Se han incluido operadores que permiten realizar a nivel de bits operaciones "and" (&), "or" (|), "not" (~), "xor" (^), desplazamiento a izquierda (<<) y desplazamiento a derecha (>>). El operador << desplaza a izquierda rellenando con ceros, mientras que el tipo de relleno realizado por >> depende del tipo de dato sobre el que se aplica: si es un dato con signo mantiene el signo, y en caso contrario rellena con ceros.

114

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos Léxicos

- Operadores

- **Operaciones de asignación:** Para realizar asignaciones se usa en C# el operador =, operador que además de realizar la asignación que se le solicita devuelve el valor asignado (el operador = es asociativo por la derecha)
 - Se ofrecen operadores de asignación compuestos para la mayoría de los operadores binarios ya vistos. Estos son: +=, -=, *=, /=, %=, &=, |=, ^=, <<= y >>=. Nótese que no hay versiones compuestas para los operadores binarios && y ||.
- **Operaciones con cadenas:** Para realizar operaciones de concatenación de cadenas se puede usar el mismo operador que para realizar sumas, ya que en C# se ha redefinido su significado para que cuando se aplique entre operandos que sean cadenas o que sean una cadena y un carácter lo que haga sea concatenarlos.
- **Operaciones de acceso a tablas:** Una **tabla** es un conjunto de ordenado de objetos de tamaño fijo. Para acceder a cualquier elemento de este conjunto se aplica el operador postfijo [] sobre la tabla para indicar entre corchetes la posición que ocupa el objeto al que se desea acceder dentro del conjunto.

115

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos Léxicos

- Operadores

- **Operador condicional:** Es el único operador incluido en C# que toma 3 operandos, y se usa así:

```
<condición> ? <expresión1> : <expresión2>
```

El significado del operando es el siguiente: se evalúa <condición> Si es cierta se devuelve el resultado de evaluar <expresión1>, y si es falsa se devuelve el resultado de evaluar <condición2>. Un ejemplo de su uso es:

```
b = (a>0)? a : 0; // Suponemos a y b de tipos enteros
```
- **Operaciones de delegados:** Un delegado es un objeto que puede almacenar en referencias a uno o más métodos y a través del cual es posible llamar a estos métodos. Para añadir objetos a un delegado se usan los operadores + y +=, mientras que para quitárselos se usan los operadores - y -=.
- **Operaciones de acceso a objetos:** Para acceder a los miembros de un objeto se usa el operador ., cuya sintaxis es: <objeto>.<miembro>

116

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos Léxicos

- Operadores

- **Operaciones con punteros:** Un puntero es una variable que almacena una referencia a una dirección de memoria. Para obtener la dirección de memoria de un objeto se usa el operador `&`, para acceder al contenido de la dirección de memoria almacenada en un puntero se usa el operador `*`, para acceder a un miembro de un objeto cuya dirección se almacena en un puntero se usa `->`, y para referenciar una dirección de memoria de forma relativa a un puntero se le aplica el operador `[]` de la forma `puntero[desplazamiento]`.

- **Operaciones de obtención de información sobre tipos:** De todos los operadores que nos permiten obtener información sobre tipos de datos el más importante es `typeof`, cuya forma de uso es:

```
typeof(<nombreTipo>) // objeto de System.Type
```

```
<expresión> is <nombreTipo>
```

```
sizeof(<nombreTipo>) // numero de bytes
```

117

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Aspectos Léxicos

- Operadores

- **Operaciones de creación de objetos:** El operador más típicamente usado para crear objetos es **new**, que se usa así: **new <nombreTipo>(<parametros>)**

- C# proporciona otro operador que también nos permite crear objetos. Éste es **stackalloc**, y se usa así:

```
stackalloc <nombreTipo>[<nElementos>]
```

- Este operador lo que hace es crear en pila una tabla de tantos elementos de tipo `<nombreTipo>` como indique `<nElementos>` y devolver la dirección de memoria en que ésta ha sido creada.
 - **stackalloc** sólo puede usarse para inicializar punteros a objetos de tipos valor declarados como variables locales. Por ejemplo:

```
int * p = stackalloc[100]; // p apunta a una tabla de 100 enteros.
```

118

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

```
class Persona
{
    string Nombre; // Campo de cada objeto Persona que almacena su nombre
    int Edad;      // Campo de cada objeto Persona que almacena su edad
    string NIF;    // Campo de cada objeto Persona que almacena su NIF

    void Cumpleaños() // Incrementa en uno la edad del objeto Persona
    {
        Edad++;
    }
    Persona (string nombre, int edad, string nif) // Constructor
    {
        Nombre = nombre;
        Edad = edad;
        NIF = nif;
    }
}
```

Ramón Hervás Lucas - Curso 2007/2008 - HyEP

119

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **Referencia al objeto actual**

- Dentro del código de cualquier método de un objeto siempre es posible hacer referencia al propio objeto usando la palabra reservada **this**. Esto puede venir bien a la hora de escribir constructores de objetos debido a que permite que los nombres que demos a los parámetros del constructor puedan coincidir nombres de los campos del objeto sin que haya ningún problema.
- Un uso más frecuente de **this** en C# es el de permitir realizar llamadas a un método de un objeto desde código ubicado en métodos del mismo objeto. Es decir, en C# siempre es necesario que cuando llamemos a algún método de un objeto precedamos al operador **.** de alguna expresión que indique cuál es el objeto a cuyo método se desea llamar, y si éste método pertenece al mismo objeto que hace la llamada la única forma de conseguir indicarlo en C# es usando **this**.
- Finalmente, una tercera utilidad de **this** es permitir escribir métodos que puedan devolver como objeto el propio objeto sobre el que el método es aplicado. Para ello bastaría usar una instrucción **return this;** al indicar el objeto a devolver

120

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **Herencia y métodos virtuales**

- Es un mecanismo que permite definir nuevas clases a partir de otras ya definidas. Las clases que derivan de otras se definen usando la siguiente sintaxis:

```
class <nombreHija>:<nombrePadre> {  
    <miembrosHija>  
}
```

- A la hora de escribir el constructor de esta clase ha sido necesario escribirlo con una sintaxis especial consistente en preceder la llave de apertura del cuerpo del método de una estructura de la forma:

```
: base(<parametrosBase>)
```

- Si en la definición del constructor de alguna clase que derive de otra no incluimos inicializador base el compilador considerará que éste es **:base()** Por ello hay que estar seguros de que si no se incluye **base** en la definición de algún constructor, el tipo padre del tipo al que pertenezca disponga de constructor sin parámetros.

121

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **Métodos Virtuales**

- Ya hemos visto que es posible definir tipos cuyos métodos se hereden de definiciones de otros tipos. Lo que ahora vamos a ver es que además es posible cambiar dicha definición en la clase hija, para lo que habría que haber precedido con la palabra reservada **virtual** la definición de dicho método en la clase padre. A este tipo de métodos se les llama **métodos virtuales**, y la sintaxis que se usa para definirlos es la siguiente:

```
virtual <tipoDevuelto> <nombreMétodo>(<parámetros>)  
{ <código> }
```

- Si en alguna clase hija quisiésemos dar una nueva definición del <código> del método, simplemente lo volveríamos a definir en la misma pero sustituyendo en su definición la palabra reservada **virtual** por **override**. Es decir, usaríamos esta sintaxis:

```
override <tipoDevuelto> <nombreMétodo>(<parámetros>)  
{ <nuevoCódigo> }
```

- Nótese que esta posibilidad de cambiar el código de un método en su clase hija sólo se da si en la clase padre el método fue definido como **virtual**. En caso contrario, el compilador considerará un error intentar redefinirlo.

122

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **Clases abstractas**

- Una clase abstracta es aquella que forzosamente se ha de derivar si se desea que se puedan crear objetos de la misma o acceder a sus miembros estáticos. Para definir una clase abstracta se antepone `abstract` a su definición:

```
public abstract class A {  
    public abstract void F();  
}  
  
abstract public class B: A {  
    public void G() {}  
}  
  
class C: B {  
    public override void F() {}  
}
```

- La utilidad de las clases abstractas es que pueden contener métodos para los que no se dé directamente una implementación sino que se deje en manos de sus clases hijas darla.)

123

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **La clase primigenia: `System.Object`**

- En .NET todos los tipos que se definan heredan implícitamente de la clase `System.Object`, por lo que dispondrán de todos los miembros de ésta. Por esta razón se dice que `System.Object` es la raíz de la jerarquía de objetos de .NET
- Métodos comunes a todos los objetos
 - **`public virtual bool Equals(object o)`**: Se usa para comparar el objeto sobre el que se aplica con cualquier otro que se le pase como parámetro (por referencia)
 - **`public virtual int GetHashCode()`**: Devuelve un código de dispersión (hash) que representa de forma numérica al objeto sobre el que el método es aplicado. `GetHashCode()` suele usarse para trabajar con tablas de dispersión, y se cumple que si dos objetos son iguales sus códigos de dispersión serán iguales, mientras que si son distintos la probabilidad de que sean iguales es ínfima.
 - **`public virtual string ToString()`**: Devuelve una representación en forma de cadena del objeto sobre el que se el método es aplicado, lo que es muy útil para depurar aplicaciones ya que permite mostrar con facilidad el estado de los objetos

Continúa...

124

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **La clase primigenia: System.Object**

- Métodos comunes
 - **protected object MemberwiseClone()**: Devuelve una copia shallow copy del objeto sobre el que se aplica. Esta copia es una copia bit a bit del mismo, por lo que el objeto resultante de la copia mantendrá las mismas referencias a otros que tuviese el objeto copiado y toda modificación que se haga a estos objetos a través de la copia afectará al objeto copiado y viceversa.
 - **public System.Type GetType()**: Devuelve un objeto de clase System.Type que representa al tipo de dato del objeto sobre el que el método es aplicado. A través de los métodos ofrecidos por este objeto se puede acceder a metadatos sobre el mismo como su nombre, su clase padre, sus miembros, etc.
 - **protected virtual void Finalize()**: Contiene el código que se ejecutará siempre que vaya a ser destruido algún objeto del tipo del que sea miembro. La implementación dada por defecto a Finalize() consiste en no hacer nada.

125

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **Polimorfismo**

- El polimorfismo es otro de los pilares fundamentales de la programación orientada a objetos. Es la capacidad de almacenar objetos de un determinado tipo en variables de tipos antecesores del primero a costa, claro está, de sólo poderse acceder a través de dicha variable a los miembros comunes a ambos tipos. Sin embargo, las versiones de los métodos virtuales a las que se llamaría a través de esas variables no serían las definidas como miembros del tipo de dichas variables, sino las definidas en el verdadero tipo de los objetos que almacenan.
- El polimorfismo es muy útil ya que permite escribir métodos genéricos que puedan recibir parámetros que sean de un determinado tipo o de cualquiera de sus tipos hijos.
- Dentro de una rutina polimórfica que admita parámetros que puedan ser de cualquier tipo, muchas veces es conveniente poder consultar en el código de la misma cuál es el tipo en concreto del parámetro que se haya pasado al método en cada llamada al mismo. Para ello C# ofrece el operador **is**, cuya forma sintaxis de uso es:
`<expresión> is <nombreTipo>`

126

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **Encapsulación**

- Ya hemos visto que la herencia y el polimorfismo eran dos de los pilares fundamentales en los que se apoya la programación orientada a objetos. Pues bien, el tercero y último es la encapsulación, que es un mecanismo que permite a los diseñadores de tipos de datos determinar qué miembros de los tipos creen pueden ser utilizados por otros programadores y cuáles no. Las principales ventajas que ello aporta son:
 - Se facilita a los programadores que vayan a usar el tipo de dato (programadores clientes) el aprendizaje de cómo trabajar con él, pues se le pueden ocultar todos los detalles relativos a su implementación interna y sólo dejarle visibles aquellos que puedan usar con seguridad. Además, así se les evita que cometan errores por manipular inadecuadamente miembros que no deberían tocar.
 - Se facilita al creador del tipo la posterior modificación del mismo, pues si los programadores clientes no pueden acceder a los miembros no visibles, sus aplicaciones no se verán afectadas si éstos cambian o se eliminan. Gracias a esto es posible crear inicialmente tipos de datos con un diseño sencillo aunque poco eficiente, y si posteriormente es necesario modificarlos para aumentar su eficiencia, ello puede hacerse sin afectar al código escrito en base a la no mejorada de tipo.

127

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Clases**

- **Encapsulación**

- **public**: Puede ser accedido desde cualquier código.
- **protected**: Desde una clase sólo puede accederse a miembros `protected` de objetos de esa misma clase o de subclases suyas.
- **private**: Sólo puede ser accedido desde el código de la clase a la que pertenece. Es lo considerado por defecto.
- **internal**: Sólo puede ser accedido desde código perteneciente al ensamblado en que se ha definido.
- **protected internal**: Sólo puede ser accedido desde código perteneciente al ensamblado en que se ha definido o desde clases que deriven de la clase donde se ha definido.

Es importante recordar que toda redefinición de un método virtual o abstracto ha de realizarse manteniendo los mismos modificadores que tuviese el método original. Es decir, no podemos redefinir un método protegido cambiando su accesibilidad por pública, pues si el creador de la clase base lo definió así por algo sería.

128

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Espacio de nombres**

- Del mismo modo que los ficheros se organizan en directorios, los tipos de datos se organizan en **espacios de nombres**.
- Por un lado estos espacios permiten tener más organizados los tipos de datos, lo que facilita su localización. De hecho, esta es la forma en que se encuentra organizada la Biblioteca de Clases
- Los espacios de nombres también permiten poder usar en un mismo programa varias clases con igual nombre si pertenecen a espacios diferentes. La idea es que cada fabricante defina sus tipos dentro de un espacio de nombres propio para que así no hayan conflictos
- Para definir un espacio de nombres se utiliza la siguiente sintaxis:

```
namespace <nombreEspacio>
```

- Aparte de definiciones de tipo, también es posible incluir como miembros de un espacio de nombres a otros espacios de nombres. Es decir, es posible anidar espacios de nombres
- Como puede resultar muy pesado tener que escribir nombres tan largos en cada referencia a tipos así definidos, en C# se ha incluido un mecanismo de importación de espacios de nombres que usa la siguiente sintaxis:

```
using <espacioNombres>;
```

129

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Espacio de Nombres**

- **Especificación de alias**

- Aún en el caso de que usemos espacios de nombres distintos para diferenciar clases con igual nombre pero procedentes de distintos fabricantes, podrían darse conflictos sin usamos sentencias **using** para importar los espacios de nombres de dichos fabricantes ya que entonces al hacerse referencia a una de las clases comunes con tan solo su nombre simple el compilador no podrá determinar a cual de ellas en concreto nos referimos
- Por ejemplo, si tenemos una clase de nombre completamente calificado A.Clase, otra de nombre B.Clase, y hacemos:

```
using A;  
using B;  
class EjemploConflicto: Clase {}
```
- Se ha incluido en C# la posibilidad de definir **alias** para cualquier tipo de dato, que son sinónimos para los mismos que se definen usando la siguiente sintaxis:

```
using <alias> = <nombreCompletoTipo>;
```
- En el ejemplo anterior podríamos usar:

```
using ClaseA = A.Clase;
```

130

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Variables y Tipos de Datos**

- **Variables:** Una variable puede verse simplemente como un almacén de objetos de un determinado tipo al que se le da un cierto nombre. Por tanto, para definir una variable sólo hay que decir cuál será el nombre que se le dará y cuál será el tipo de datos que podrá almacenar, lo que se hace con la siguiente sintaxis:

`<tipoVariable> <nombreVariable>;`

- **Tablas unidimensionales:** Una **tabla unidimensional** es un tipo especial de variable que es capaz de almacenar en su interior y de manera ordenada uno o varios datos de un determinado tipo

`<tipoDatos>[] <nombreTabla> = new <tipoDatos>[] { <valores>};`

- **Tablas dentadas:** Una **tabla dentada** no es más que una tabla cuyos elementos son a su vez tablas, pudiéndose así anidar cualquier número de tablas. Ejemplo:

`int[][] tablaDentada = new int[][] {new int[] {1,2}, new int[] {3,4,5}};`

- **Tablas Multidimensionales y Tablas Mixtas**

131

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Variables y Tipos de Datos**

- **La clase System.Array:** En realidad, todas las tablas que definamos, sea cual sea el tipo de elementos que contengan, son objetos que derivan de System.Array. Es decir, van a disponer de todos los miembros que se han definido para esta clase, entre los que son destacables:

- **Length:** Campo de sólo lectura que informa del número total de elementos que contiene la tabla.

- **Rank:** Campo de sólo lectura que almacena el número de dimensiones de la tabla. Obviamente si la tabla no es multidimensional valdrá 1.

- **GetLength(int dimensión):** Método que devuelve el número de elementos de la dimensión especificada. Las dimensiones se indican empezando a contar desde cero.

- **CopyTo(Array destino, int posición):** Copia todos los elementos de la tabla sobre la que es aplicada en la que se le pasa como primer parámetro a partir de la posición de la misma indicada como segundo parámetro.

132

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Variables y tipos de datos

- Cadenas de texto:

- Una cadena de texto no es más que una secuencia de caracteres Unicode. En C# se representan mediante objetos del tipo de dato llamado **string**, que no es más que un alias del tipo **System.String** incluido en la Biblioteca de Clases.
 - Hay que tener en cuenta que el tipo **string** es un tipo referencia, por lo que en principio la comparación entre objetos de este tipo debería comparar sus direcciones de memoria. Sin embargo, para hacer para hacer más intuitivo el trabajo con cadenas, en C# se ha modificado el operador de igualdad (al igual que el operador + para concatenación) para que cuando se aplique entre cadenas se considere que sus operandos son iguales sólo si son lexicográficamente equivalentes y no si referencian al mismo objeto en memoria.
 - Si se quisiese comparar cadenas por referencia habría que optar por una de estas dos opciones: compararlas con **Object.ReferenceEquals()** o convertirlas en **objects** y luego compararlas con **==**. Por ejemplo:

```
Console.WriteLine(Object.ReferenceEquals(cadena1, cadena2));
```

```
Console.WriteLine( (object) cadena1 == (object) cadena2);
```

133

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Variables y Tipos de Datos

- Cadenas de Texto

- **Length**. El acceso a las cadenas se hace de manera similar a como si de tablas de caracteres se tratase: su "campo" Length almacenará el número de caracteres que la forman y para acceder a sus elementos se utiliza el operador **[]**. Sin embargo, hay que señalar una diferencia importante respecto a la forma en que se accede a las tablas: las cadenas son inmutables, lo que significa que no es posible modificar los caracteres que las forman. Si se desea trabajar con cadenas modificables puede usarse Sytem.Text.StringBuilder.
 - **int IndexOf(string subcadena)**
 - **int LastIndexOf(string subcadena)**
 - **string Insert(int posición, string subcadena)**
 - **string Remove(int posición, int número)**
 - **string Replace(string aSustituir, string sustituta)**
 - **string Substring(int posición, int número)**
 - **string ToUpper()** y **string ToLower()**

134

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Variables y Tipos de Datos**

- **Constantes**

- Una **constante** es una variable cuyo valor puede determinar el compilador durante la compilación y puede aplicar optimizaciones derivadas de ello.
 - Para que esto sea posible se ha de cumplir que el valor de una constante no pueda cambiar durante la ejecución, por lo que el compilador informará con un error de todo intento de modificar el valor inicial de una constante.
 - Su sintaxis es:

const <tipoConstante> <nombreConstante> = <valor>;

- **Variables de sólo lectura**

- Dado que hay ciertos casos en los que resulta interesante disponer de la capacidad de sólo lectura que tienen las constantes pero no es posible usarlas debido a las restricciones que hay impuestas sobre su uso, en C# también se da la posibilidad de definir variables que sólo puedan ser leídas. Para ello se usa la siguiente sintaxis:

readonly <tipoConstante> <nombreConstante> = <valor>;

135

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Estructuras**

- Una **estructura** es un tipo especial de clase pensada para representar objetos ligeros. Es decir, que ocupen poca memoria y deban ser manipulados con velocidad, como objetos que representen puntos, fechas, etc.
 - A diferencia de una clase y fielmente a su espíritu de "ligereza", una estructura no puede derivar de ningún tipo y ningún tipo puede derivar de ella. Por estas razones sus miembros no pueden incluir modificadores relativos a herencia.
 - Otra diferencia es que sus variables no almacenan referencias a zonas de memoria dinámica donde se encuentran almacenados objetos sino directamente referencian a objetos. Las clases son **tipos referencia** y las estructuras son **tipos valor**.
 - De lo anterior se deduce que la asignación entre objetos de tipos estructuras es mucho más lenta que la asignación entre objetos de clases, ya que se ha de copiar un objeto completo y no solo una referencia.
 - Todas las estructuras derivan implícitamente del tipo **System.ValueType**, que a su vez deriva de la clase primigenia **System.Object**. **ValueType** tiene los mismos miembros que su padre, y la única diferencia entre ambos es que en **ValueType** se ha redefinido **Equals()** de modo que devuelva **true** si los objetos comparados tienen el mismo valor en todos sus campos y **false** si no.
 - Respecto a la implementación de la igualdad en los tipos definidos como estructuras, también es importante tener muy en cuenta que el operador **==** no es en principio aplicable a las estructuras que defina el programador.

136

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Estructuras

- Ejemplo

```
struct Point
{
    public int x, y;
    public Point(int x, int y) {
        this.x = x; this.y = y; }
}
```

Si usamos este tipo en un código como el siguiente:

```
Punto p = new Punto(10,10);
Punto p2 = p;
p2.x = 100;
Console.WriteLine(p.x);
```

¿Qué mostrara como salida en la Consola?

137

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- Enumeraciones

- Una **enumeración** o **tipo enumerado** es un tipo especial de estructura en la que los literales de los valores que pueden tomar sus objetos se indican explícitamente al definirla

- Sintaxis:

```
enum <nombreEnumeración> : <tipoBase>
{ <literales> }
```

- No sólo facilitan la escritura y lectura del código sino que también pueden ser usados por herramientas de documentación, depuradores u otras aplicaciones para sustituir números mágicos y mostrar textos muchos más legibles.

- Ejemplo

```
enum Tamaño:int
{ Pequeño = 0, Mediano = 1, Grande = 2 }
```

- Las variables de tipos enumerados se definen como cualquier otra variable (sintaxis <nombreTipo> <nombreVariable>) Por ejemplo:

```
Tamaño t;
Tamaño t = Tamaño.Grande;
```

138

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Enumeraciones**

- **static Type getUnderlyingType(Type enum):** Devuelve un objeto System.Type con información sobre el tipo base de la enumeración representada por el objeto System.Type que se le pasa como parámetro
- **string ToString(string formato):** Cuando a un objeto de un tipo enumerado se le aplica el método ToString() heredado de object lo que se muestra es una cadena con el nombre del literal almacenado en ese objeto
- **static object[] GetValues(Type enum):** Devuelve una tabla con los valores de todos los literales de la enumeración representada por el objeto System.Type que se le pasa como parámetro
- **static string GetName(Type enum, object valor):** Devuelve una cadena con el nombre del literal de la enumeración representada por enum que tenga el valor especificado en valor.
- **static bool isDefined (Type enum, object valor):** Devuelve un booleano que indica si algún literal de la enumeración indicada tiene el valor indicado

139

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Interfaces**

- Una **interfaz** es la definición de un conjunto de métodos para los que no se da implementación, sino que se les define de manera similar a como se definen los métodos abstractos. Es más, una interfaz puede verse como una forma especial de definir clases que sólo cuenten con miembros abstractos.
- Como las clases abstractas, las interfaces son tipos referencia, no puede crearse objetos de ellas sino sólo de tipos que deriven de ellas, y participan del polimorfismo.
- Sin embargo, también tiene muchas diferencias:
 - Es posible definir tipos que deriven de más de una interfaz. con las interfaces se permite la herencia múltiple porque no pueden ocurrir conflictos debido a que las interfaces no incluyen código.
 - Aunque las estructuras no pueden heredar clases, sí pueden hacerlo de interfaces
 - Todo tipo que derive de una interfaz ha de dar una implementación de todos los miembros que hereda de esta, y no como ocurre con las clases abstractas donde es posible no darla si se define como abstracta también la clase hija.
 - Las interfaces sólo pueden tener como miembros métodos normales, eventos, propiedades e indicadores; pero no pueden incluir definiciones de campos, operadores, constructores o destructores

140

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Interfaces**

- Definición: La sintaxis general que se sigue a la hora de definir una interfaz es:

```
<modificadores> interface <nombre>:<interfacesBase>
{ <miembros> }
```

- Los <miembros> de las interfaces pueden ser
 - **Métodos:** <tipoRetorno> <nombreMétodo>(<parámetros>);
 - **Propiedades:** <tipo> <nombrePropiedad> {set; get;} Los bloques get y set pueden intercambiarse y puede no incluirse uno de ellos (propiedad de sólo lectura o de sólo escritura según el caso), pero no los dos.
 - **Indizadores:** <tipo> this[<índices>] {set; get;} Al igual que las propiedades, los bloques set y get pueden intercambiarse y obviarse uno de ellos al definirlos.
 - **Eventos:** event <delegado> <nombreEvento>;

141

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Excepciones**

- Las **excepciones** son el mecanismo recomendado en la plataforma .NET para la propagación de errores que se produzcan durante la ejecución de las aplicaciones
- Básicamente una excepción es un objeto derivado de **System.Exception** que se genera cuando en tiempo de ejecución se produce algún error y que contiene información sobre el mismo
- Las excepciones proporcionan las siguientes ventajas:
 - **Claridad:** El uso de códigos especiales para informar de error suele dificultar la legibilidad del fuente en tanto que se mezclan las instrucciones propias de la lógica del mismo con las instrucciones propias del tratamiento de los errores que pudiesen producirse durante su ejecución.
 - **Más información:** A partir del valor de un código de error puede ser difícil deducir las causas del mismo y conseguirlo muchas veces implica tenerse que consultar la documentación que proporcionada sobre el método que lo provocó, que puede incluso que no especifique claramente su causa.
 - **Tratamiento Asegurado:** Cuando se usan excepciones siempre se asegura que el programador trate toda excepción que pueda producirse o que, si no lo hace, se aborte la ejecución de la aplicación mostrándose un mensaje indicando dónde se ha producido el error

142

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Excepciones**

- **La clase System.Exception**

- **string Message {virtual get;}**: Contiene un mensaje descriptivo de las causas de la excepción. Por defecto este mensaje es una cadena vacía ("")
- **Exception InnerException {virtual get;}**: Si una excepción fue causada como consecuencia de otra, esta propiedad contiene el objeto System.Exception que representa a la excepción que la causó. Si se desea obtener la última excepción de la cadena es mejor usar el método **virtual Exception GetBaseException()**
- **string StackTrace {virtual get;}**: Contiene la pila de llamadas a métodos que se tenía en el momento en que se produjo la excepción.
- **string Source {virtual get; virtual set;}**: Almacena información sobre cuál fue la aplicación u objeto que causó la excepción.
- **MethodBase TargetSite {virtual get;}**: Almacena cuál fue el método donde se produjo la excepción.
- **string HelpLink {virtual get;}**: Contiene una cadena con información sobre cuál es la URI donde se puede encontrar información sobre la excepción..

143

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Excepciones**

- **Lanzamiento de excepciones**

- Para informar de un error no basta con crear un objeto del tipo de excepción apropiado, sino que también hay pasárselo al mecanismo de propagación de excepciones del CLR. A esto se le llama **lanzar la excepción**, y para hacerlo se usa la siguiente instrucción:
throw <objetoExcepciónALanzar>;
- Por ejemplo, para lanzar una excepción de tipo **DivideByZeroException** se podría hacer:
throw new DivideByZeroException();
- Si el objeto a lanzar vale **null**, entonces se producirá una **NullReferenceException** que será lanzada en vez de la excepción indicada en la instrucción **throw**.

144

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Excepciones**

- **Captura de excepciones**

- Si se desea tratar la excepción hay que encerrar la división dentro de una instrucción try con la siguiente sintaxis:

```
try
    <instrucciones>
catch (<excepción1>)
    <tratamiento1>
catch (<excepción2>)
    <tratamiento2>
...
finally
    <instruccionesFinally>
```

- Si durante la ejecución de las <instrucciones> se lanza una excepción de tipo <excepción1> (o alguna subclase suya) se ejecutan las instrucciones <tratamiento1>, si fuese de tipo <excepción2> se ejecutaría <tratamiento2>, y así hasta que se encuentre una cláusula catch que pueda tratar la excepción producida. Si no se encontrase ninguna y la instrucción try estuviese anidada dentro de otra, se miraría en los catch de su try padre y se repetiría el proceso.
 - El bloque **finally** es opcional, y si se incluye ha de hacerlo tras todas los bloques catch. Las <instruccionesFinally> de este bloque se ejecutarán tanto si se producen excepciones en <instrucciones> como si no.

145

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Controles de Usuario**

- Los controles de usuario proporcionan un medio para crear y reutilizar interfaces gráficas de usuario.
 - Un control de usuario es esencialmente un componente con una representación visual. Como tal, puede constar de uno o más controles de formularios Windows Forms, componentes o bloques de código.
 - Pueden extender su funcionalidad mediante la validación de la entrada del usuario, la modificación de las propiedades de presentación o la ejecución de otras tareas requeridas por su autor.
 - Los controles de usuario pueden incluirse en formularios Windows Forms de la misma manera que los demás controles.
 - Existen dos tipos de controles de usuario:
 - **Controles personalizados:** Controles que muestran la IU mediante llamadas a un objeto Graphics en el evento Paint. Normalmente, los controles personalizados se derivan de Control. Un control Chart es un ejemplo de control personalizado. Existe compatibilidad en tiempo de diseño limitada en lo que se refiere a la creación de controles personalizados.
 - **Controles compuestos:** Controles que se componen de otros controles. Los controles de usuario se derivan de UserControl. Un control que muestra una dirección de cliente utilizando el control TextBox es un ejemplo de control de usuario. Existe compatibilidad en tiempo de diseño completa en lo que se refiere a la creación de controles de usuario con el diseñador de formularios de Windows de Visual Studio .NET.

146

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Documentación XML**

- La documentación de los tipos de datos creados siempre ha sido una de las tareas más pesadas y aburridas a las que un programador se ha tenido que enfrentar durante un proyecto, lo que ha hecho que muchas veces se escriba de manera descuidada y poco concisa o que incluso que no se escriba en absoluto.
- Sin embargo, escribirla es una tarea muy importante sobre todo en un enfoque de programación orientada a componentes en tanto que los componentes desarrollados muchas veces van a reutilizados por otros o, incluso, por uno mismo.
- Para facilitar la pesada tarea de escribir la documentación, el compilador de C# es capaz de generarla automáticamente a partir de los comentarios que el programador escriba en los ficheros de código fuente.
- El hecho de que la documentación se genere a partir de los fuentes permite evitar que se tenga que trabajar con dos tipos de documentos por separado (fuentes y documentación) que deban actualizarse simultáneamente para evitar inconsistencias.
- El compilador genera la documentación en XML con la idea de que sea fácilmente legible para cualquier aplicación. Para facilitar su legibilidad a humanos bastaría añadirle una hoja de estilo XSL o usar alguna aplicación específica encargada de leerla y mostrarla de una forma más cómoda para humanos.

147

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Documentación XML**

- Sintaxis: Los comentarios de documentación XML se escriben como comentarios normales de una línea pero con las peculiaridades de que su primer carácter ha de ser siempre / y de que su contenido ha de estar escrito en XML ya que será insertado por el compilador en el fichero XML de documentación que genera. Por tanto, son comentarios de la forma:
`/// <textoXML>`
- Estos comentarios han preceder las definiciones de los elementos a documentar.
- En <textoXML> el programador puede incluir cualesquiera etiquetas con el significado, contenido y atributos que considere oportunos, ya que en principio el compilador no las procesa sino que las incluye tal cual en la documentación que genera dejando en manos de las herramientas encargadas de procesar dicha documentación la determinación de si se han usado correctamente.
- Sin embargo, el compilador comprueba que los comentarios de documentación se coloquen donde deberían y que contengan XML bien formado.

148

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Documentación XML**

- Aunque en principio los atributos de las etiquetas no tienen ningún significado predeterminado para el compilador, hay una excepción: el atributo **cref** siempre va a tener un significado concreto consistente en forzarlo a comprobar cuando vaya a generar la documentación si existe el elemento cuyo nombre indique y, si no es así, hacerle producir un mensaje de aviso (su nombre viene de "check reference")
- Etiquetas de uso genéricos
 - **<summary>**: Su contenido se utiliza para indicar un resumen sobre el significado del elemento al que precede. Cada vez que en VS.NET se use el operador . para acceder a algún miembro de un objeto o tipo se usará esta información para mostrar sobre la pantalla del editor de texto un resumen acerca de su utilidad.
 - **<remarks>**: Su contenido indica una explicación detallada sobre el elemento al que precede. Se recomienda usar <remarks> para dar una explicación detallada de los tipos de datos y <summary> para dar una resumida de cada uno de sus miembros.
 - **<example>**: Su contenido es un ejemplo sobre cómo usar el elemento al que precede.
 - **<seealso>**: Se usa para indicar un elemento cuya documentación guarda alguna relación con la del elemento al que precede.
 - **<permission>**: Se utiliza para indicar qué permiso necesita un elemento para poder funcionar.

149

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Documentación XML**

- **Etiquetas relativas a métodos**

- **<param>**: Permite documentar el significado de un parámetro de un método. En su propiedad name se indica el nombre del parámetro a documentar y en su contenido se describe su utilidad.
- **<paramref>**: Se usa para referenciar a parámetros de métodos. No tiene contenido y el nombre del parámetro referenciado se indica en su atributo name.
- **<returns>**: Permite documentar el significado del valor de retorno de un método, indicando como contenido suya una descripción sobre el mismo.

```
/// <summary>
/// Método que muestra por pantalla un texto con un determinado color
/// </summary>
/// <param name="texto"> Texto a mostrar </param>
/// <param name="color">
/// Color con el que mostrar el <paramref name="texto"/> indicado
/// </param>
/// <returns> Indica si el método se ha ejecutado con éxito o no </summary>

bool MuestraTexto(string texto, Color color) {...}
```

150

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Documentación XML**

- **Etiquetas relativas a propiedades**

- El uso más habitual de una propiedad consiste en controlar la forma en que se accede a un campo privado, por lo que esta se comporta como si almacenase un valor. Mediante el contenido de la etiqueta **<value>** es posible describir el significado de ese valor.

- **Etiquetas relativas a excepciones**

- Para documentar el significado de un tipo definido como excepción puede incluirse un resumen sobre el mismo como contenido de una etiqueta de documentación **<exception>** que preceda a su definición. El atributo **cref** de ésta suele usarse para indicar la clase de la que deriva la excepción definida. Por ejemplo:

```
/// <exception cref="System.Exception">  
/// Excepción de ejemplo creada por Josan  
/// </exception>
```

```
class JosanExcepción: Exception {}
```

151

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Documentación XML**

- **Etiquetas relativas al formato**

- **<see>**: Se utiliza para indicar hipervínculos a otros elementos de la documentación generada.
 - **<code>** y **<c>**: Ambas etiquetas se usan para delimitar textos han de ser considerarse fragmentos de código fuente. La diferencia entre ellas es que **<code>** se recomienda usar para fragmentos multilínea y **<c>** para los de una única línea.
 - **<para>**: Se usa para delimitar párrafos dentro del texto contenido en otras etiquetas, considerándose que el contenido de cada etiqueta **<para>** forma parte de un párrafo distinto. Generalmente se usa dentro de etiquetas **<remarks>**, ya que son las que suelen necesitar párrafos al tener un contenido más largo.
 - **<list>**: Se utiliza para incluir listas y tablas como contenido de otras etiquetas. Todo uso de esta etiqueta debería incluir un atributo **type** que indique el tipo de estructura se desea definir según tome uno de los siguientes valores:
 - **bullet**: Indica que se trata de una lista no numerada
 - **number**: Indica que se trata de una lista numerada
 - **table**: Indica que se trata de una tabla

152

Herramientas y Entornos de Programación

Tema 3. Entornos de Desarrollo. Lenguaje C#

- **Ejercicio propuesto: Diseñar un ecosistema donde podemos encontrar:**
 - Varios tipos de animales (gatos, perros, leones, ratones,...).
 - Algunos de estos animales son depredadores (comen para obtener “energía”) y otros presas (son comidos y proporcionan “energía”), y otros pueden actuar tanto como depredadores como presas.
 - Los animales en general, pueden moverse aunque cada uno de una forma y velocidad distinta. También pueden cazar pero sólo los depredadores. Por el contrario, las presas pueden huir pero no cazar.
 - Diseñar este ecosistema empleando herencia, polimorfismo, interfaces y/o abstracción.
 - El modelo debe representar el ecosistema de la manera más fiel posible
 - Se debe maximizar la reutilización de código
 - Evitar incongruencias: una presa no puede cazar a un depredador